

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 2-1: Linear Classifier with Softmax

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Aug/27/2026



Administration

- JetStream2: updated user guide
- Github repo is ready:
 - <https://github.com/MinghanLi/CSCI598H-Applied-Computer-Vision-for-Engineering-Systems>
- Code Section
 - Time and location

Time	Classroom	Topics
Sept/11 3pm-4pm	Marquez 204	Access to GPU / Python / Numpy
Sept/25 12pm-1pm	Marquez 204	CNNs / Transformers
Oct/09	TBD	Segmentation / LLMs
Oct/30	TBD	Diffusion Model / NeRF
Nov/20	TBD	Engineering Applications



**Scan QR code
to add your
available time!**

Launch Instance



Details*

Source*

Flavor*

Networks

Network Ports

Security Groups

Key Pair

Configuration

Server Groups

Scheduler Hints

Metadata

Select the security groups to launch the instance in.

▼ Allocated 1

Displaying 1 item

Name	Description
> default1	

Default1

Displaying 1 item

▼ Available 3

Select one or more

Displaying 3 items

Name	Description
> rh	
> JN	
> default	Default security group

Displaying 3 items

× Cancel

< Back

Next >

Launch Instance

Add Key in SSH Config

Step 0: change the permission (too often to use)

```
minghanli@mac ~$ chmod 600 /path/to/jetstream2_cs_course.pem
```

Step 1: open ssh config in terminal

```
minghanli@mac ~$ vim ~/.ssh/config
```

Step 2: paste the config

```
vim (vim) #1
Host mailab
  HostName 149.165.150.227
  User ubuntu
  IdentityFile ~/.jetstream2_cs_course.pem

Host minghan_test
  HostName 149.165.173.33
  User ubuntu
  IdentityFile ~/.jetstream2_cs_course.pem
```

CSCI 598H Course Overview



Weeks 2-4

Weeks 5-11

Weeks 12-16

Module 1

Deep Learning Basics

- **Linear Classifier and Optimization**
- CNNs and BackPropagation
- RNNs, Attention, Transformers

Module 2

Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

Module 3

Engineering Systems

- Robotics
- Earth science
- Medical imaging
- Energy

Recap from Last Week

W1: Course overview

Technical Landscape

Recognition & Localization & Motion

Classification

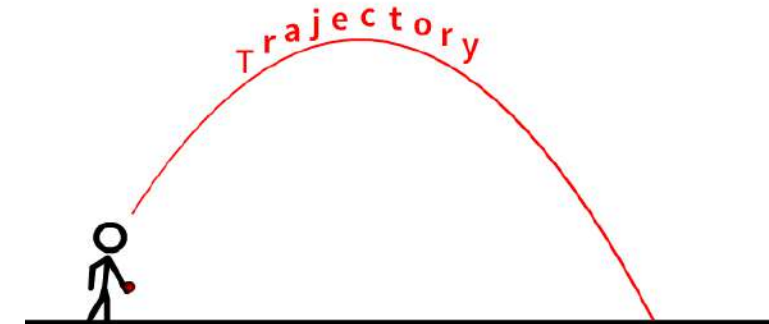


Apple, **Burro**, Cat, Horse ...

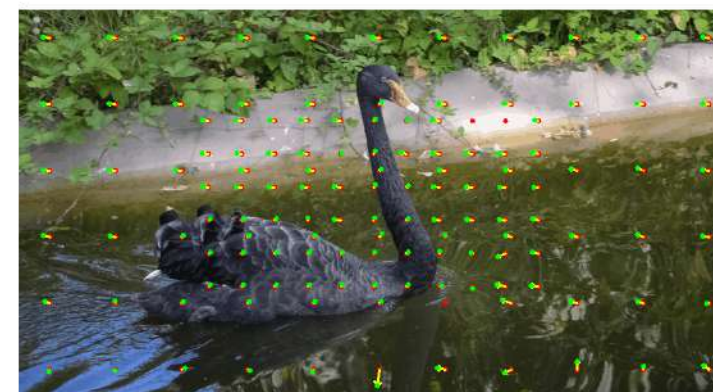
Object Detection



Segmentation



Point Tracking



Object Tracking



Foundation Models



A cat

A dog

A nuclear submarine



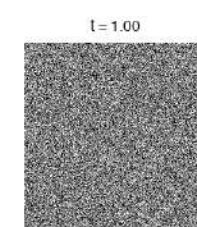
CLIP (Contrastive Language-Image Pre-training)

Predict the next token

Input text	Tokenize	LLM	Output probabilities	Sample																
Weights & Biases is the	[1135, 2337, 1222, 8436, 1386, 318, 262]		<table><tr><td>...</td><td>...</td></tr><tr><td>We</td><td>0.001</td></tr><tr><td>ights</td><td>0.001</td></tr><tr><td>&</td><td>0.001</td></tr><tr><td>the</td><td>0.001</td></tr><tr><td>machine</td><td>0.15</td></tr><tr><td>learning</td><td>0.001</td></tr><tr><td>platform</td><td>0.2</td></tr></table>	...	...	We	0.001	ights	0.001	&	0.001	the	0.001	machine	0.15	learning	0.001	platform	0.2	machine
...	...																			
We	0.001																			
ights	0.001																			
&	0.001																			
the	0.001																			
machine	0.15																			
learning	0.001																			
platform	0.2																			

LLMs

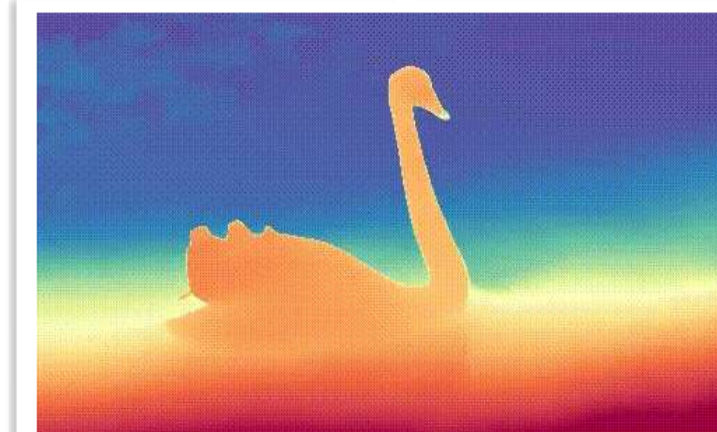
Reverse (Denosing)



Diffusion Models

Geometry & 3D Vision

Depth Estimation



Multi-view Matching



3D Object Reconstruction



3D Scene Reconstruction

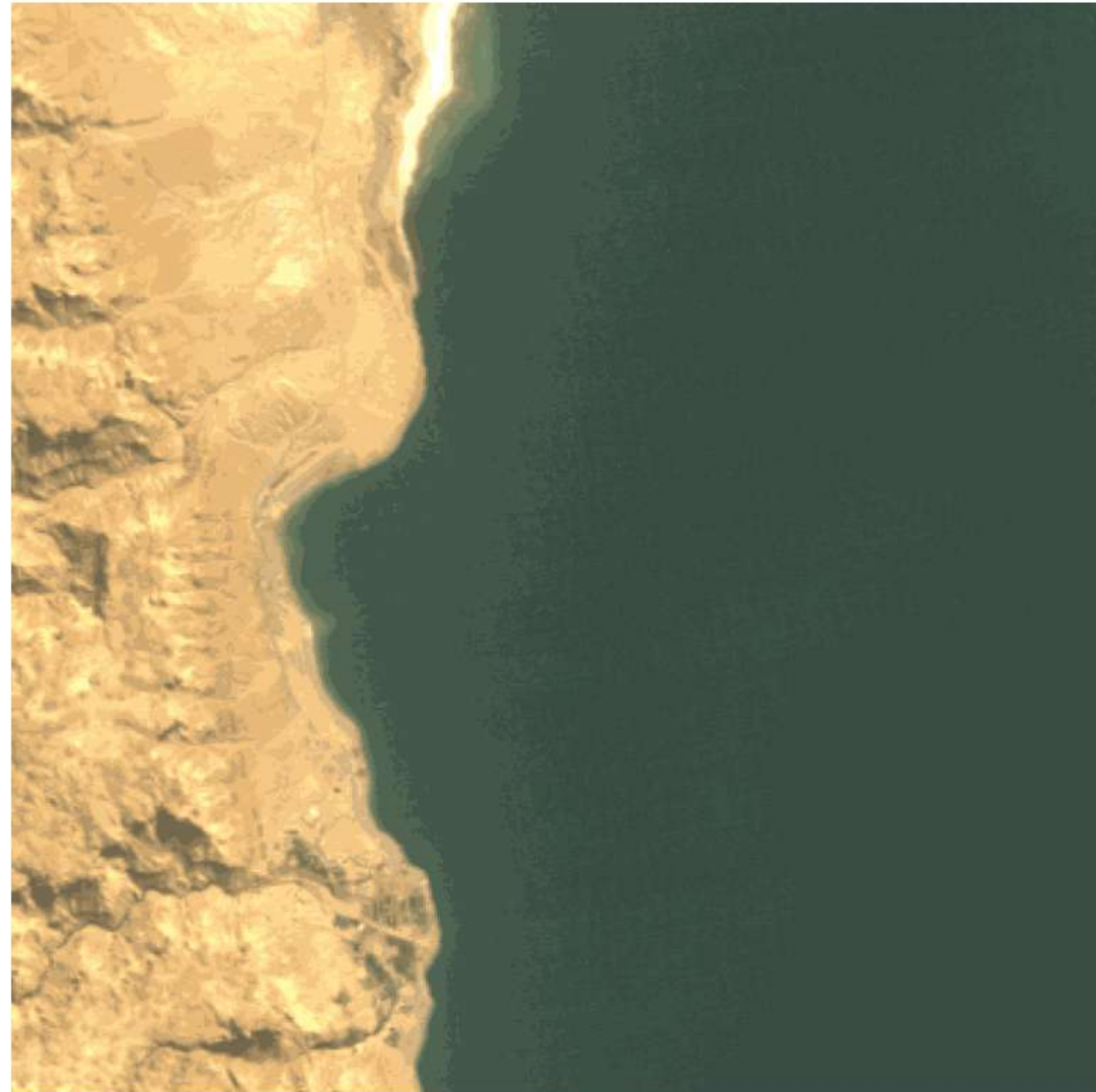


What / where is in the world?

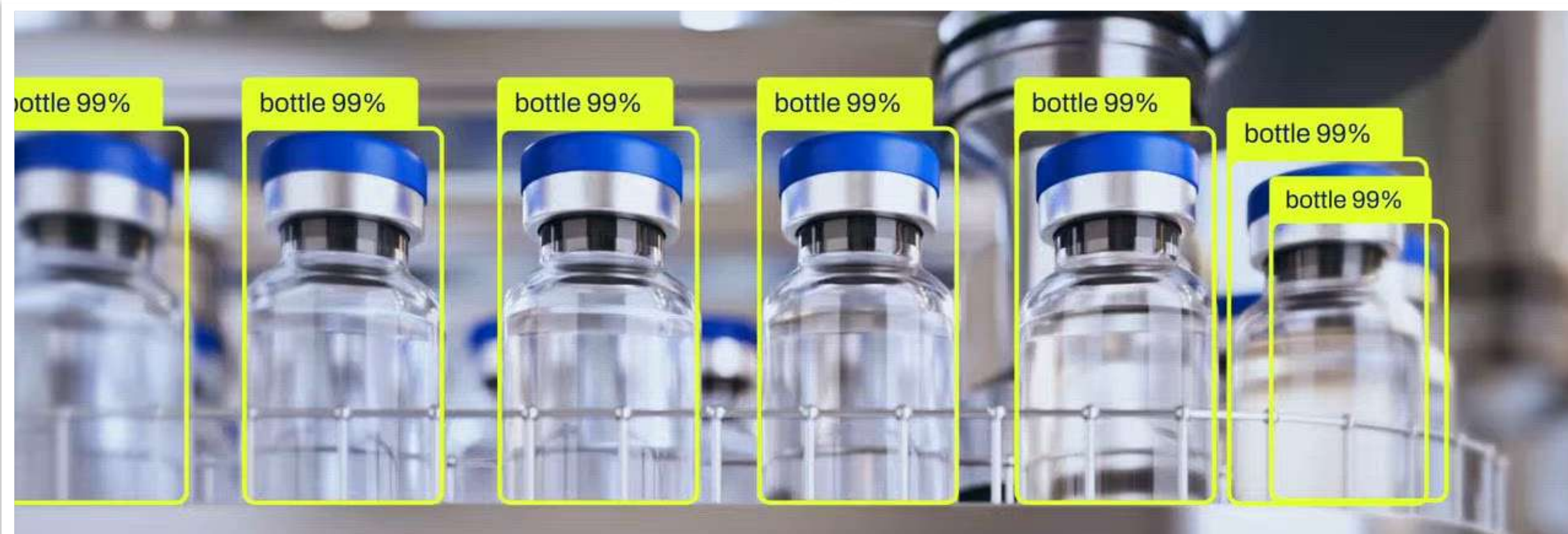
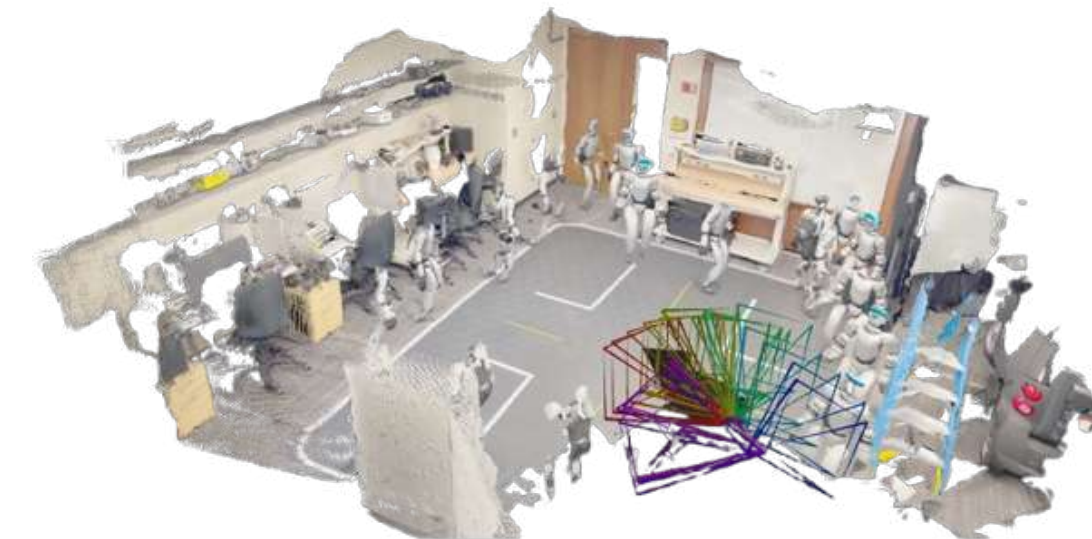
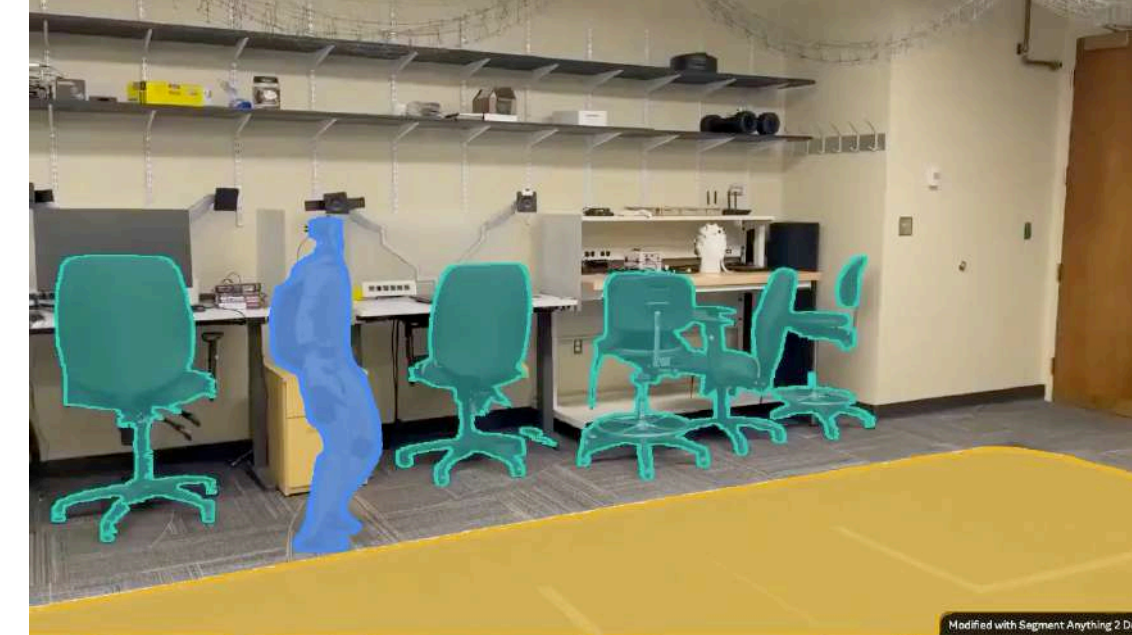
What is the 3D structure of the world?

Engineering Applications

Year: 1984

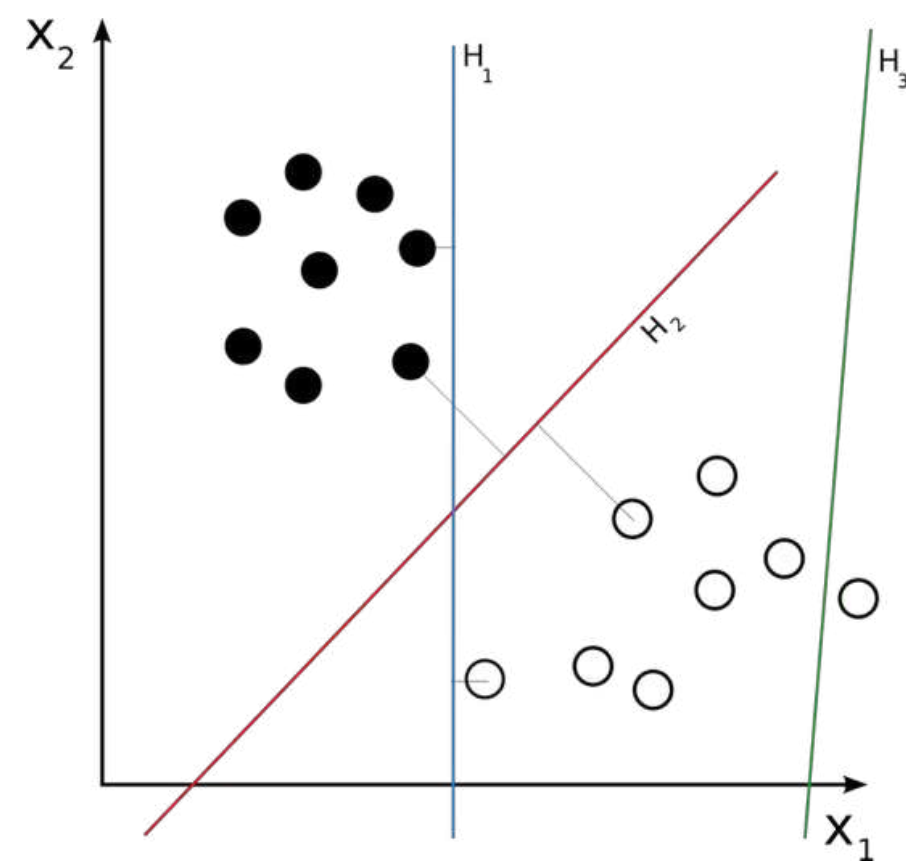


[Link](#) The Dead Sea drying up

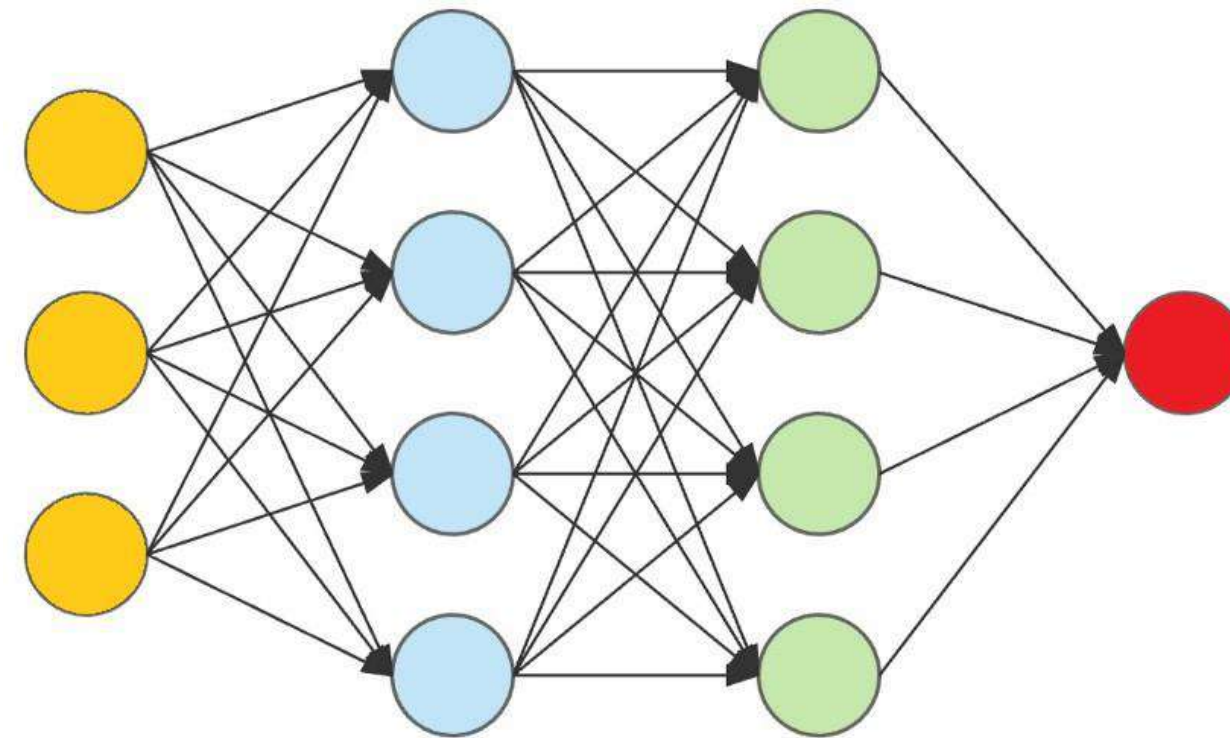


[Link](#)

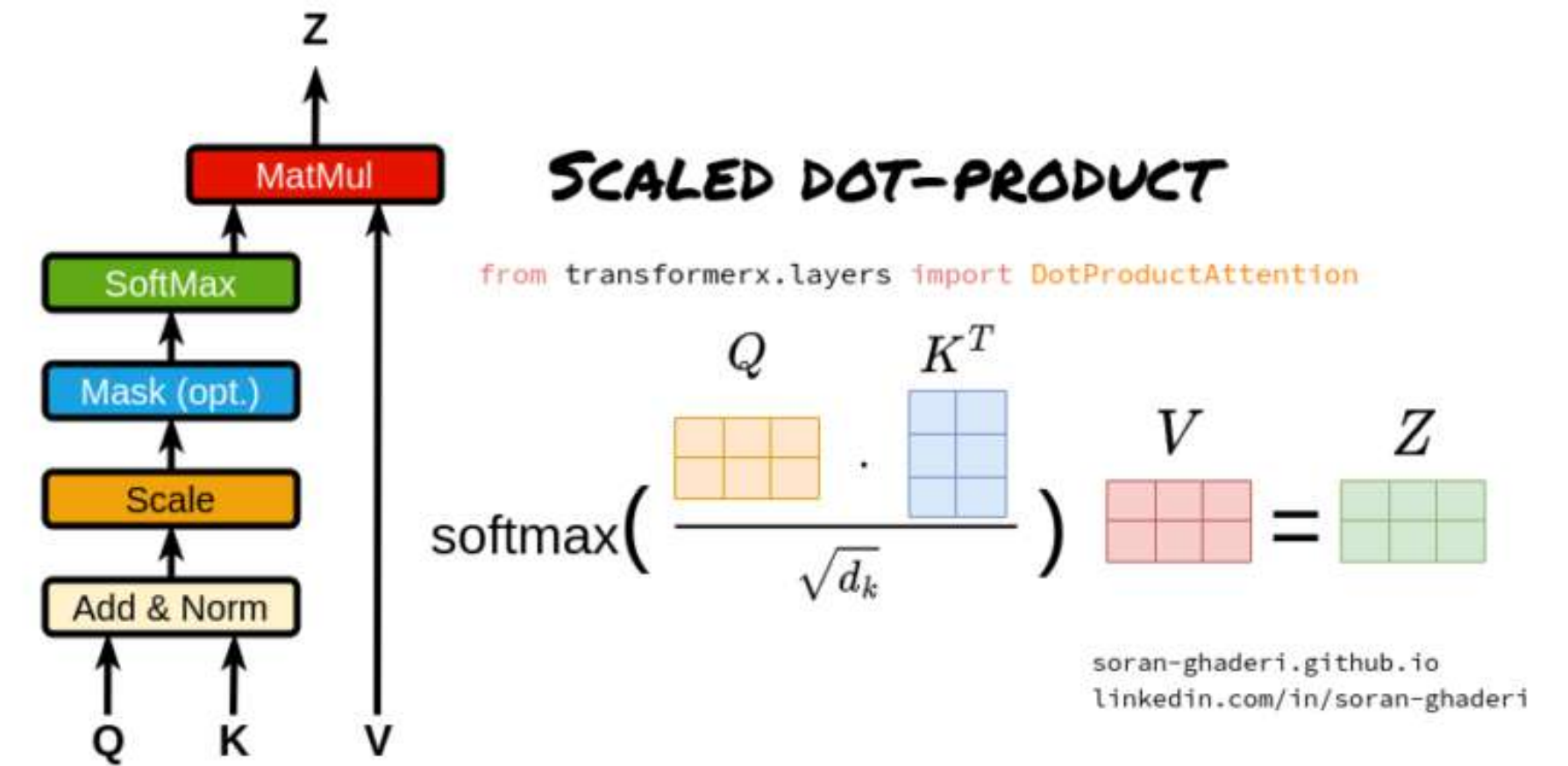
Deep Learning Basics Overview



Linear Classifier



Neural Networks

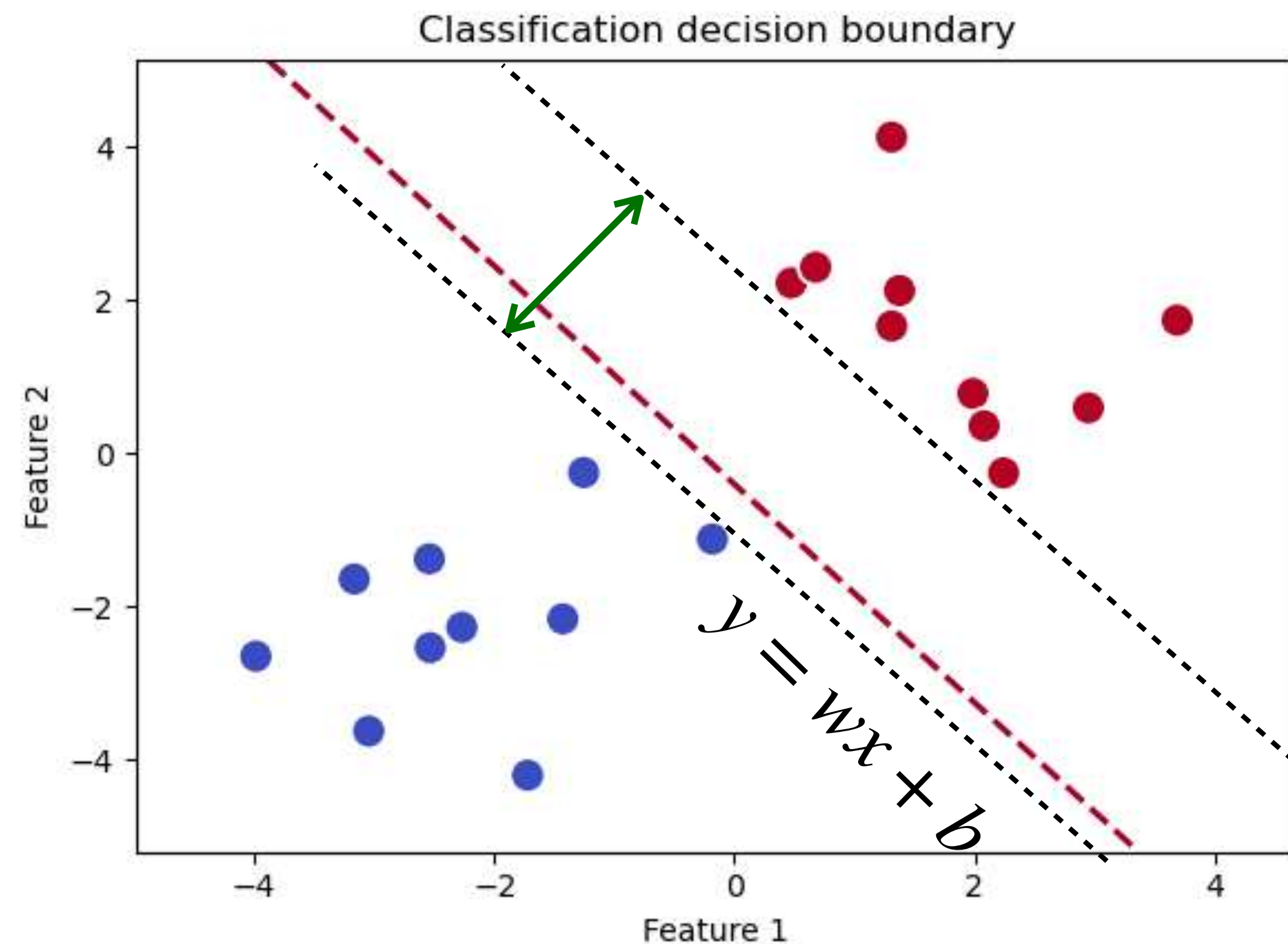


Attention mechanism / Transformer

Week 2 Agenda

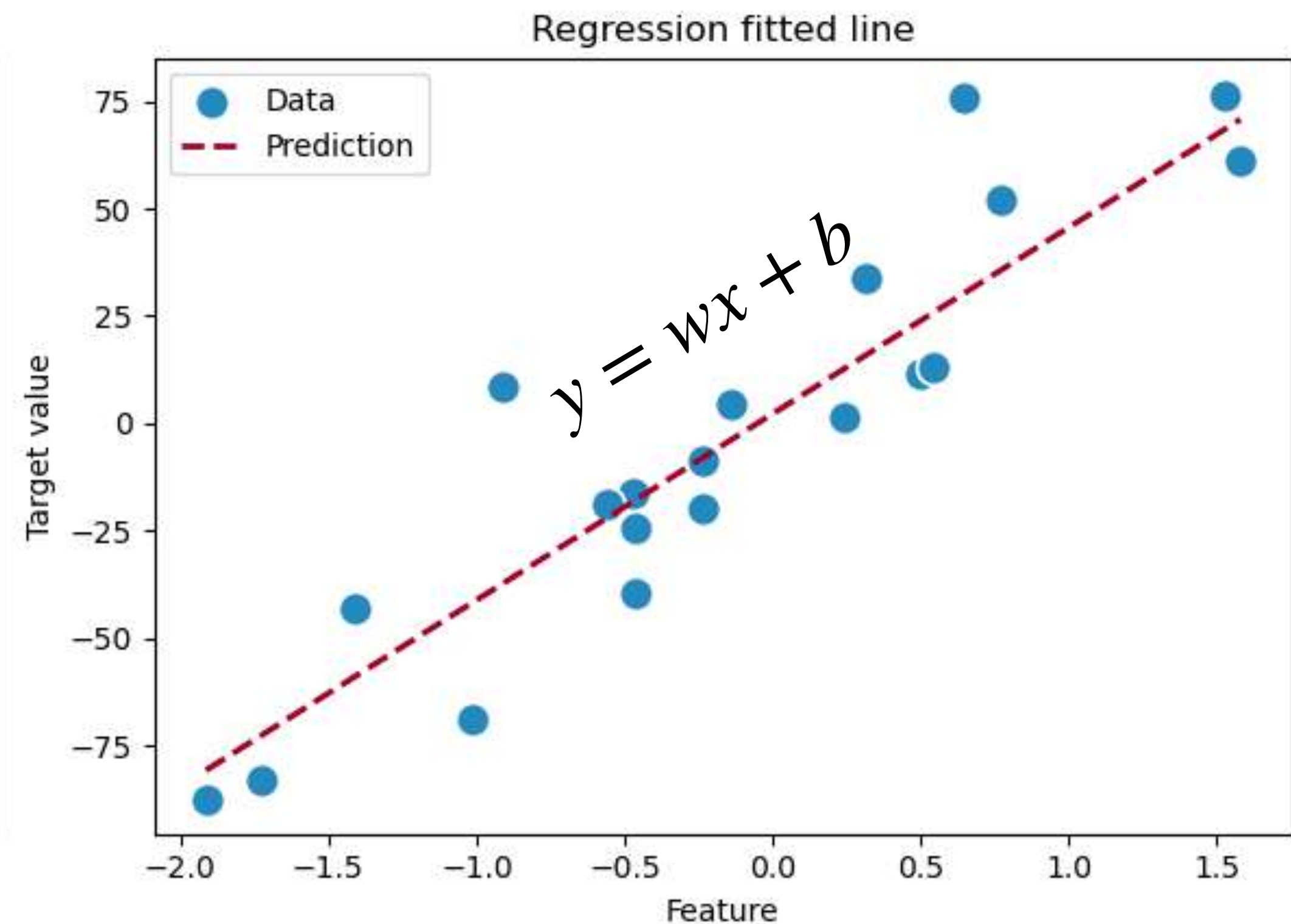
- Image Classification: Linear Classifier with Softmax
- Regularization and Optimization

ML Fundamentals: Classification & Regression



Goal: Draw a **decision boundary** to separate data into distinct classes

cat or dog



Goal: Fit a best-fit line or curve through data points.

temperature, depth, or price

Image Classification: A core task in Computer Vision



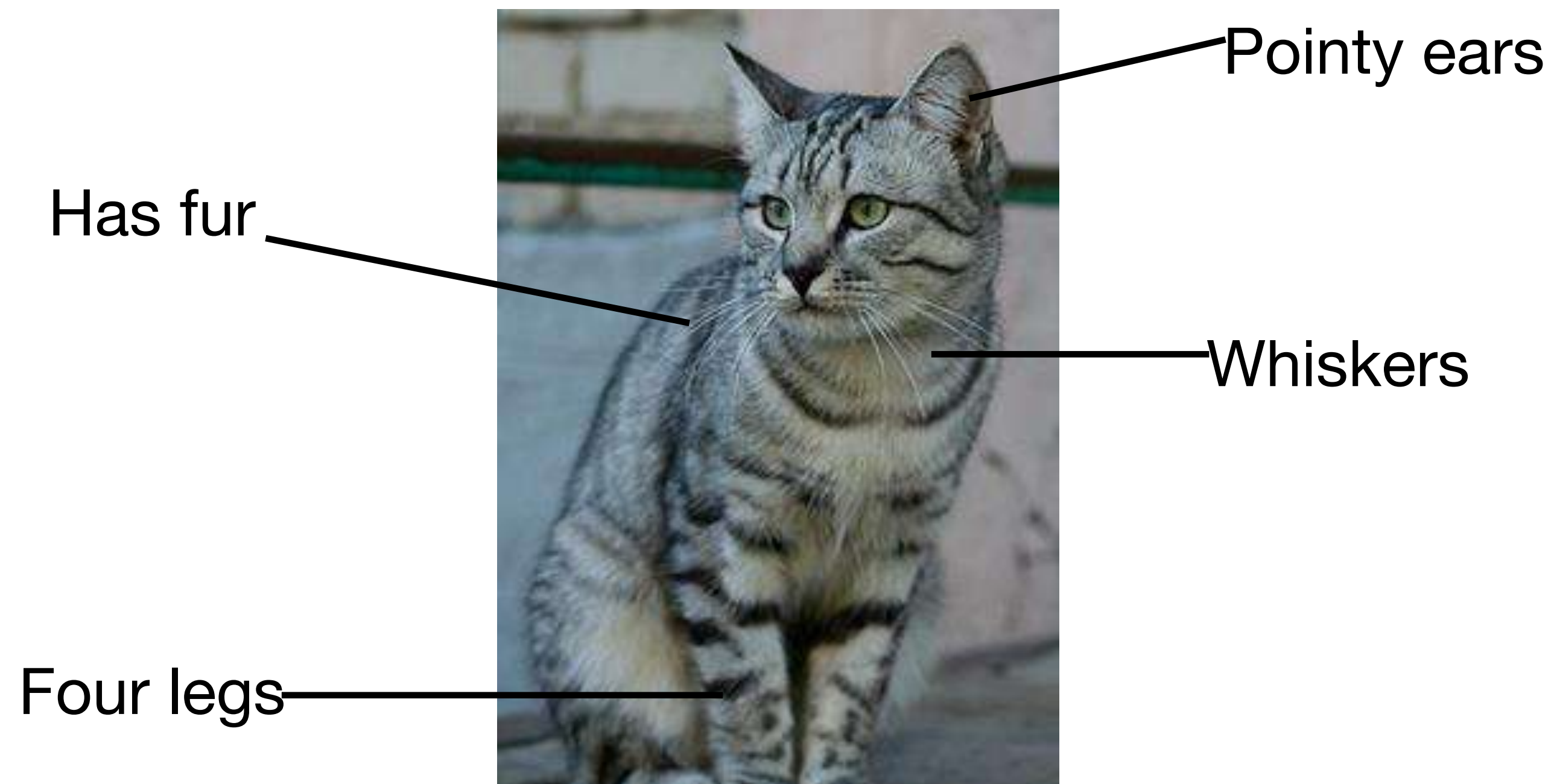
[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}



cat

Image Classification: What makes a cat a cat?



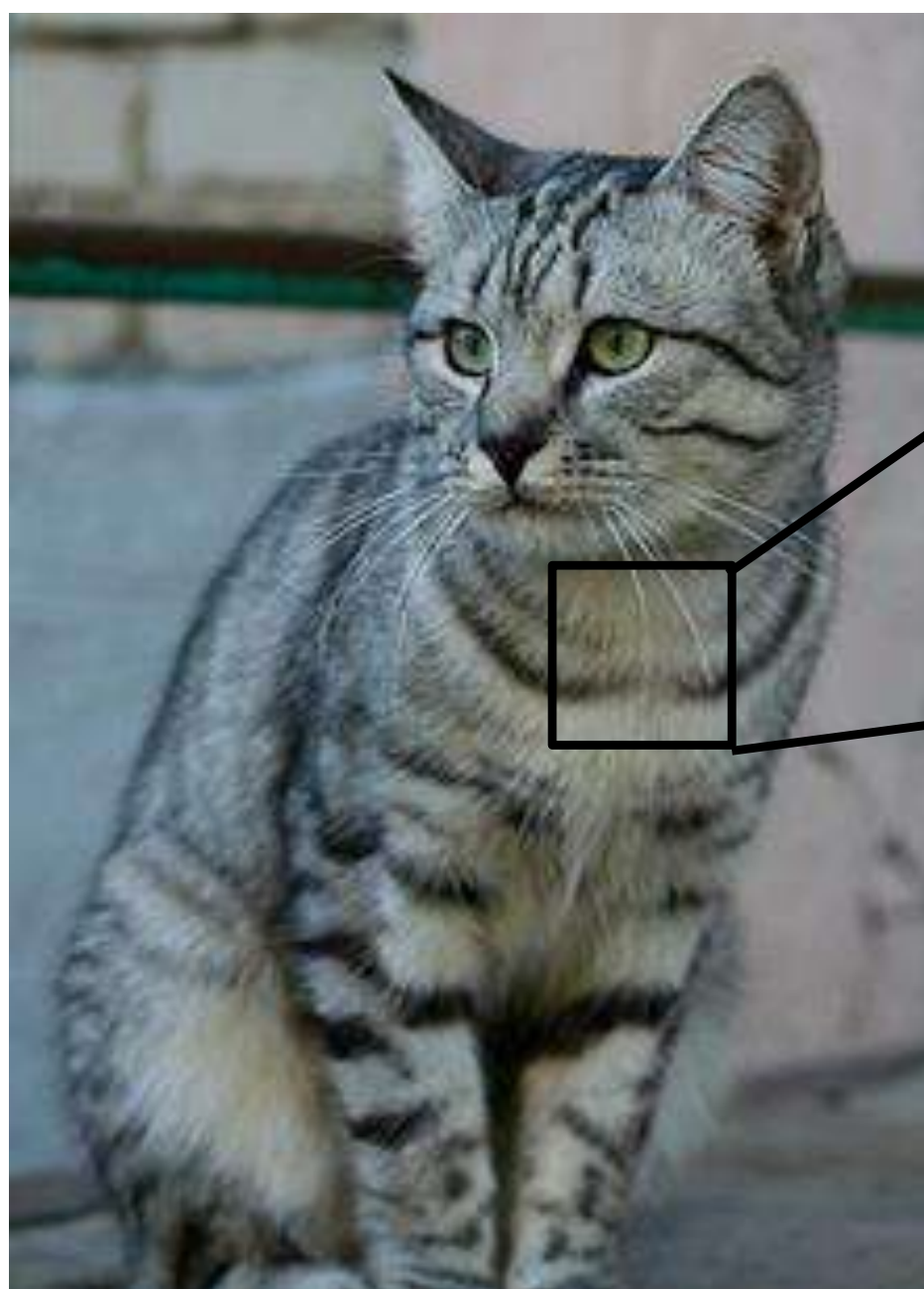
Rules-based classifier

```
# Hard-code: Check if the image is a car
def is_cat(image):
    if not has_fur(image):
        return False
    if not has_four_legs(image):
        return False
    if not pointy_ears(image):
        return False
    if not whiskers(image):
        return False
    return True
```

Poor generalization !

Real world is complex.

The Problem: Semantic Gap



[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

```
[105 112 100 111 104 99 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]  
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]  
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]  
[128 137 144 140 109 95 86 78 62 65 63 63 68 73 86 101]  
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 100 147 131 118 113 114 113 109 106 95 77 80]  
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]  
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]  
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]  
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]  
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]  
[157 170 157 120 93 86 114 132 112 97 69 55 78 82 99 94]  
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]  
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]
```

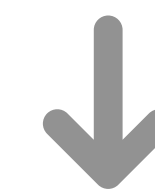
What the computer sees

An image is a tensor of integers
between $[0, 255]$:

e.g. $800 \times 600 \times 3$
(3 channels RGB)



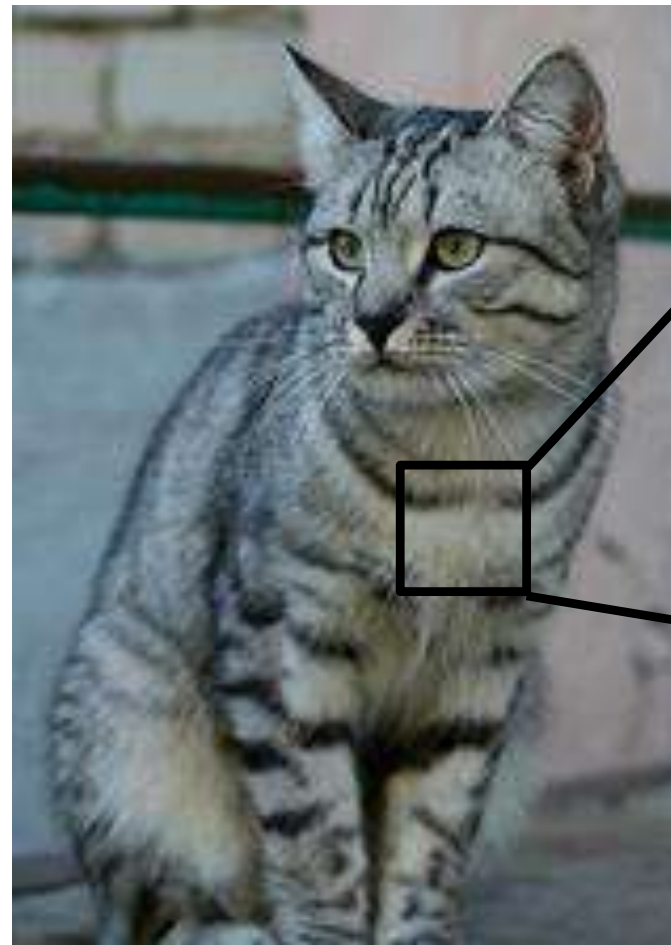
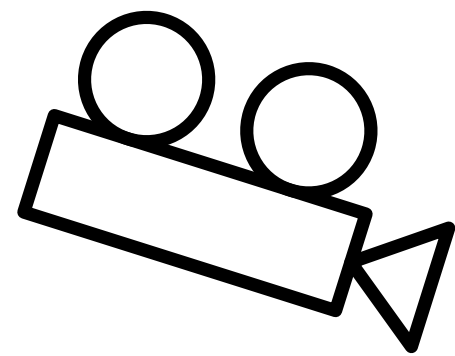
Input dimension:
1,440,000



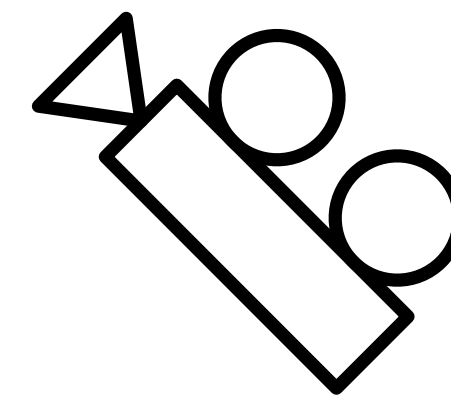
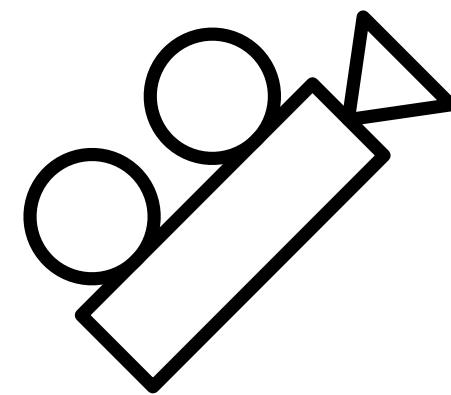
**Curse of dimensionality
(Wiki)**

when the number of datapoints is small (in a suitably defined sense) relative to the intrinsic dimension of the data.

Challenges: Viewpoint variation



1040	132	100	111	104	99	106	95	90	103	112	119	104	87	93	872	
1	91	98	102	106	104	79	88	103	99	105	123	130	118	105	94	853
1	76	85	98	105	120	105	87	96	95	99	115	112	106	103	99	853
1	99	85	81	93	120	131	127	100	95	98	102	99	96	93	101	942
1086	92	61	64	88	91	88	85	101	107	100	98	75	84	88	953	
1116	100	85	55	55	69	84	54	64	87	112	129	98	76	84	913	
1133	137	147	103	65	81	88	65	52	54	74	84	102	93	85	923	
1120	137	144	100	109	95	86	78	62	65	93	83	68	73	86	1013	
1125	133	140	137	119	121	117	94	65	79	88	95	54	64	72	983	
1127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	843	
1115	114	109	129	158	140	131	118	113	109	100	92	74	63	72	783	
1	89	83	88	97	100	107	111	116	113	114	113	109	106	93	77	883
1	63	77	86	81	77	79	102	123	117	115	117	125	125	130	115	873
1	62	69	82	80	78	71	68	101	124	126	119	101	107	114	111	1193
1	63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	1103
1	87	85	71	87	106	95	69	45	76	130	126	107	92	94	105	1123
1110	87	82	86	117	123	116	66	41	53	95	93	89	95	102	1073	
1104	146	112	88	82	120	124	104	76	48	45	66	88	101	102	1093	
1157	130	157	120	93	86	114	132	117	97	60	55	70	82	99	140	
1130	120	134	101	139	100	109	118	121	134	114	87	65	53	69	863	
1120	112	96	117	110	104	120	115	104	107	102	83	87	91	72	793	
1123	107	96	88	87	112	113	149	122	100	104	76	88	107	112	993	
1122	123	102	88	82	86	94	117	143	140	153	102	50	70	92	1073	
1127	104	140	103	71	56	78	83	93	103	110	130	102	61	68	8433	



All pixels change when
the camera moves!

[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

A good classifier should keep the same label even when the view changes.

Any Others Challenges?



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain

Illumination



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image by [jonsson](#) is license d under [CC-BY 2.0](#)



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [sarebear](#) is licensed under [CC-BY 2.0](#)



This image by [Tom Thai](#) is license d under [CC-BY 2.0](#)

Occlusion

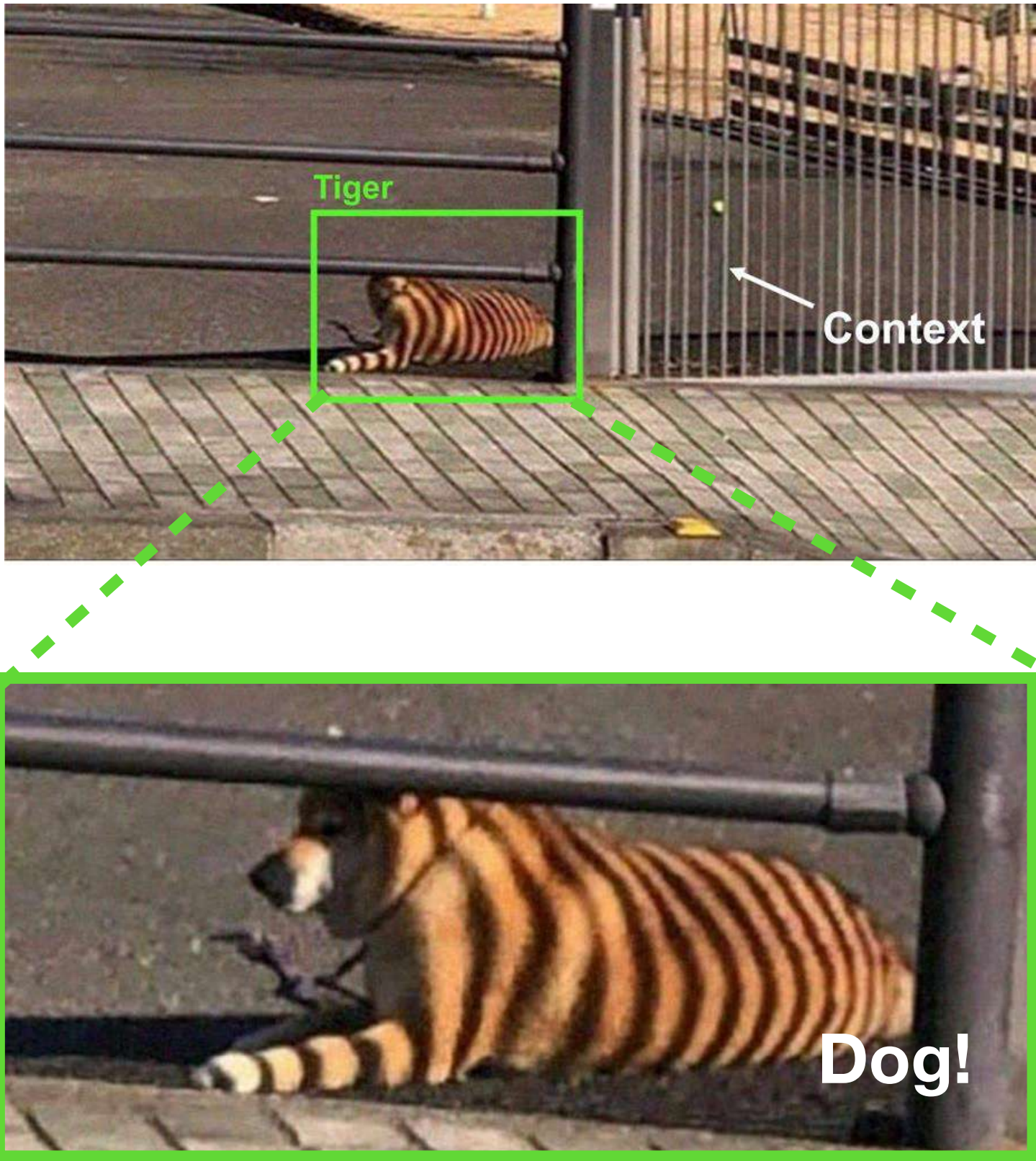
Background Clutter

Deformation

Challenges



Intraclass variation



Context

Modern Computer Vision: Data-driven Algorithms



Machine Learning: **Data-Driven** Approach

1. Collect a dataset of images and labels **Ground-truth labels**
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images **Good generalization !**

Example training set

```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

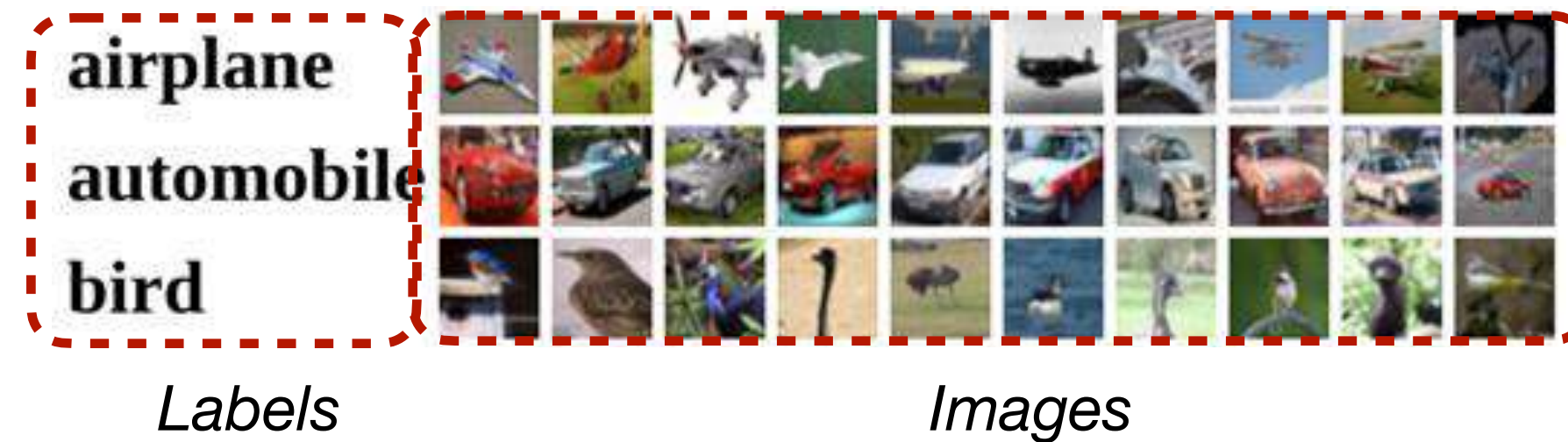
airplane
automobile
bird
cat
deer



Modern Computer Vision: **Data-driven** Algorithms

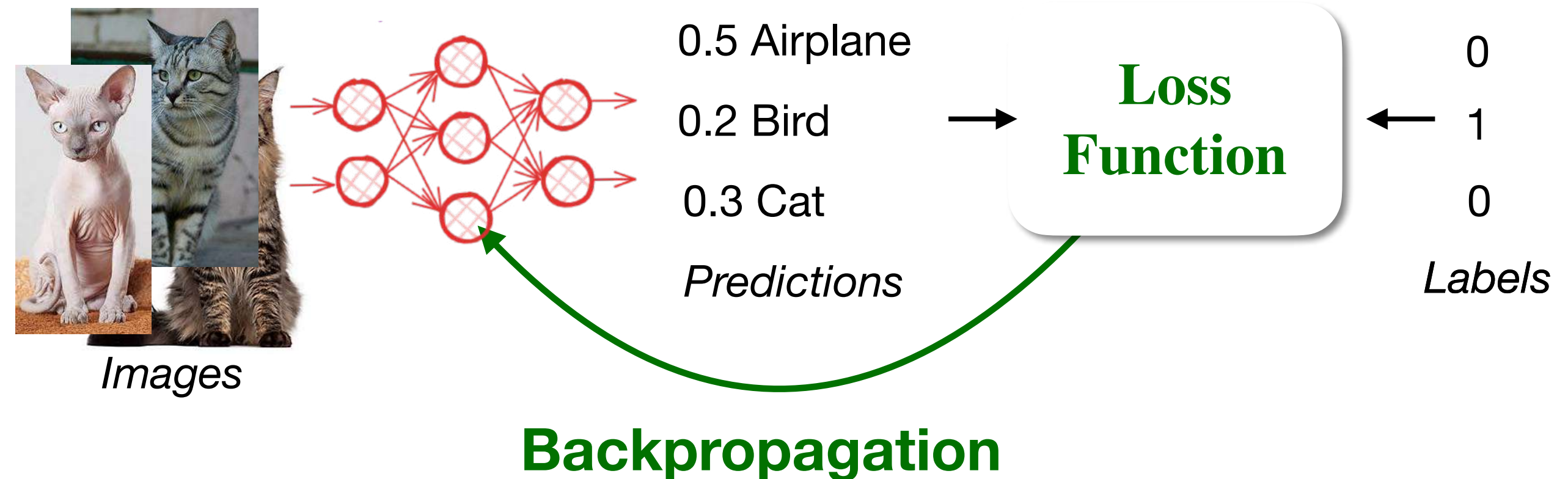
Data

Example training set Human annotations



Model

GPUs

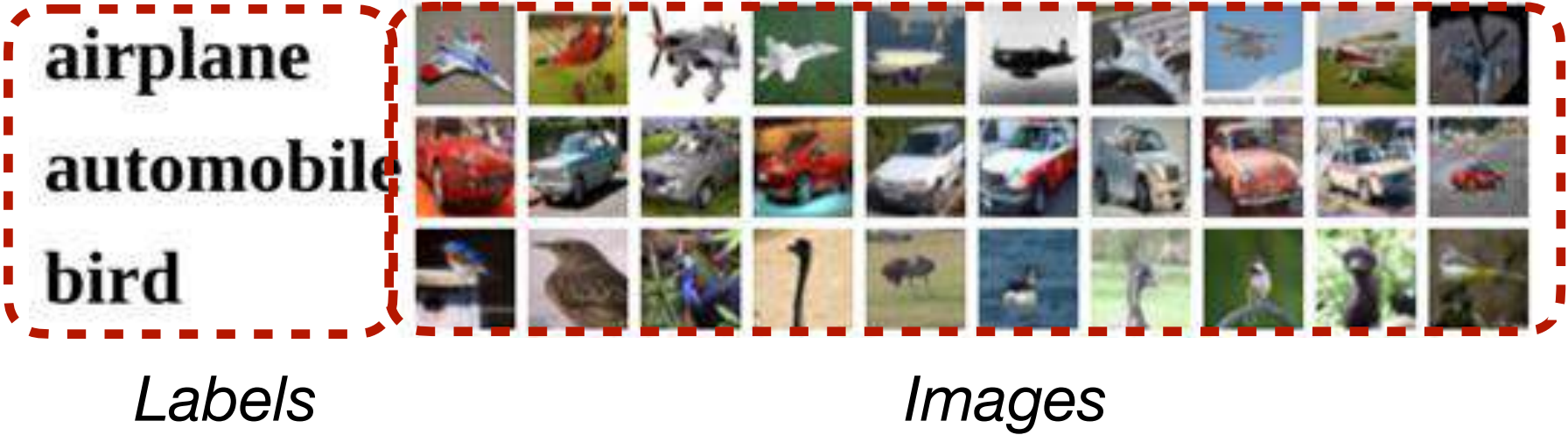


Model Training

Modern Computer Vision: Data-driven Algorithms

Data

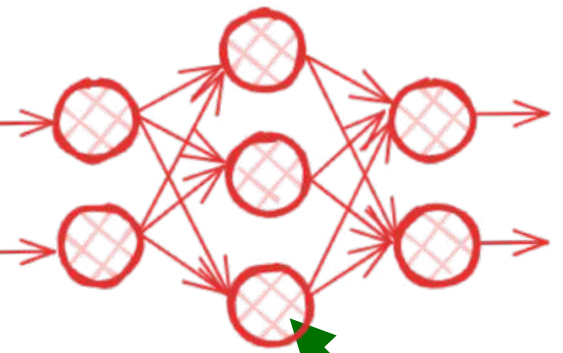
Example training set Human annotations



Model

GPUs

Training



0.5 Airplane
0.2 Bird
0.3 Cat
Predictions

Loss
Function

0
1
0
Labels

Backpropagation

Inference
new images



0.0 Airplane
0.1 Bird
0.9 Cat



Logits

Linear Classifier

Softmax Normalization + Cross-Entropy Loss + Regularization

https://d2l.ai/chapter_linear-classification/softmax-regression.html#classification

Parametric Approach

Image



Array of 32x32x3 numbers
(3072 numbers total)



$f(x, W)$



W

parameters
or weights



10 numbers giving
class scores

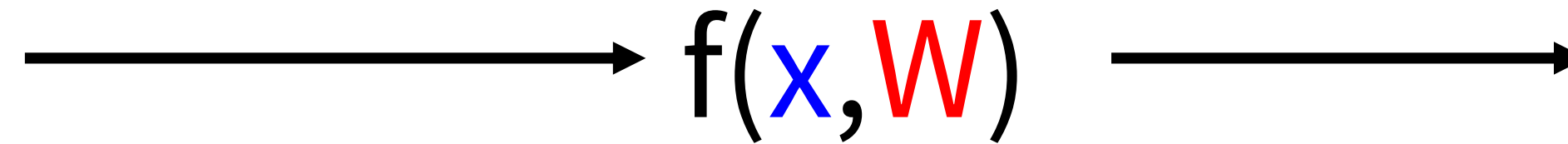
Parametric Approach: Linear Classifier

$$f(x, W) = W x$$

Image



Array of 32x32x3 numbers
(3072 numbers total)



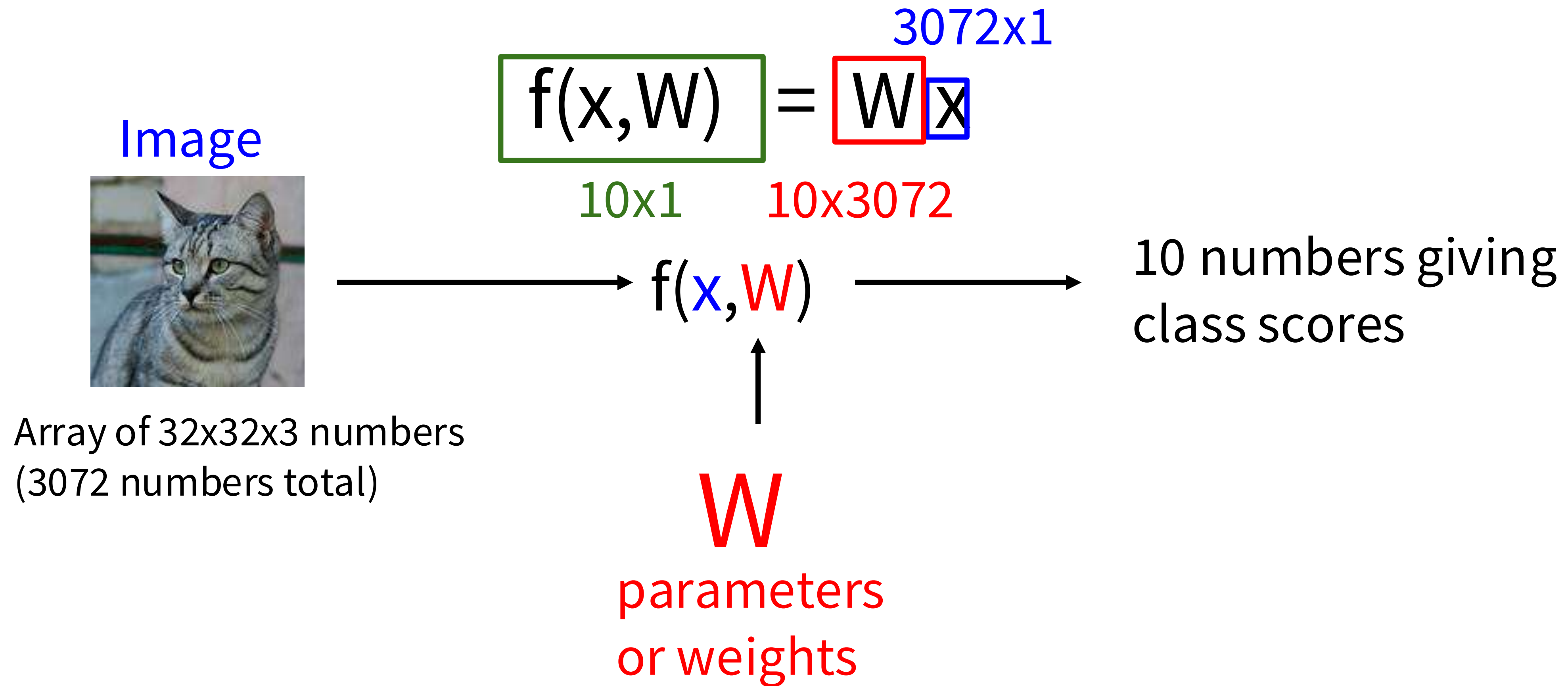
$f(x, W)$

10 numbers giving
class scores

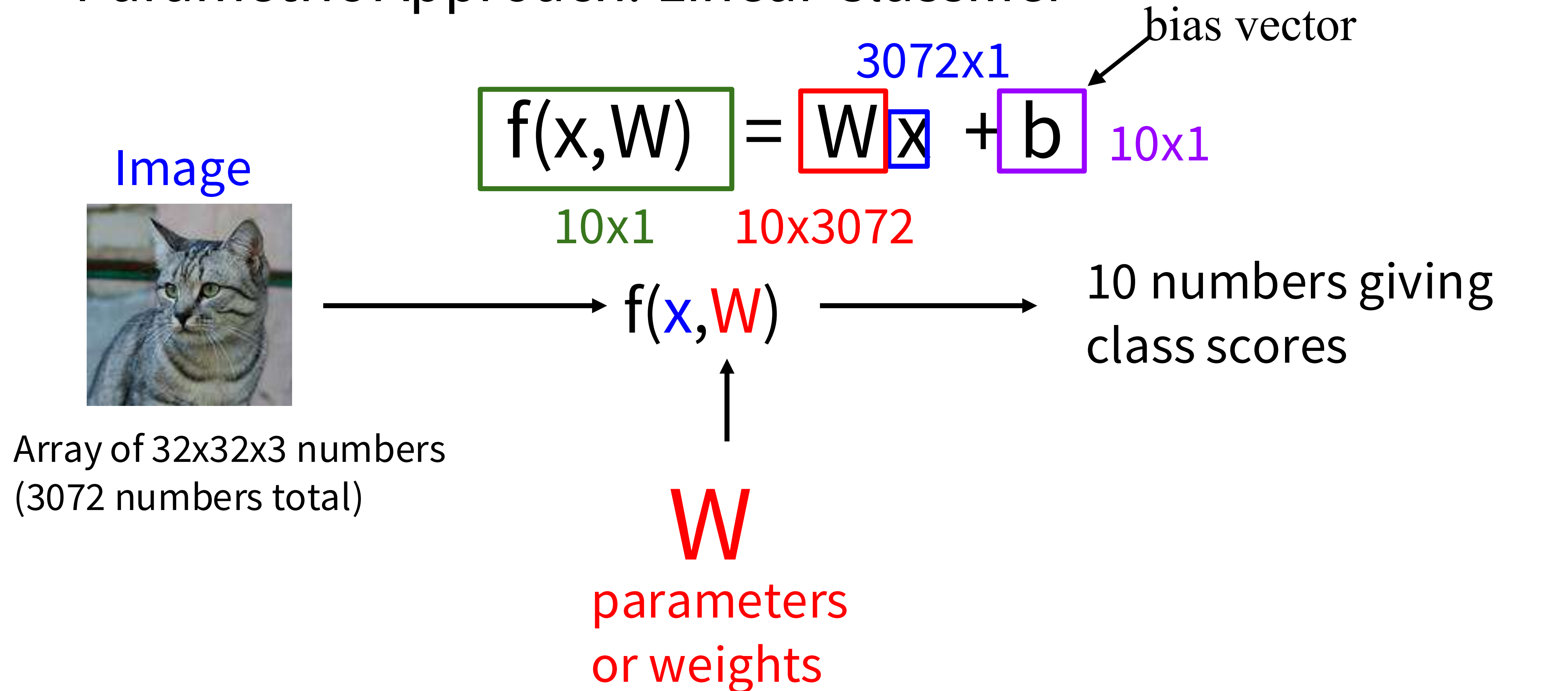
W

parameters
or weights

Parametric Approach: Linear Classifier



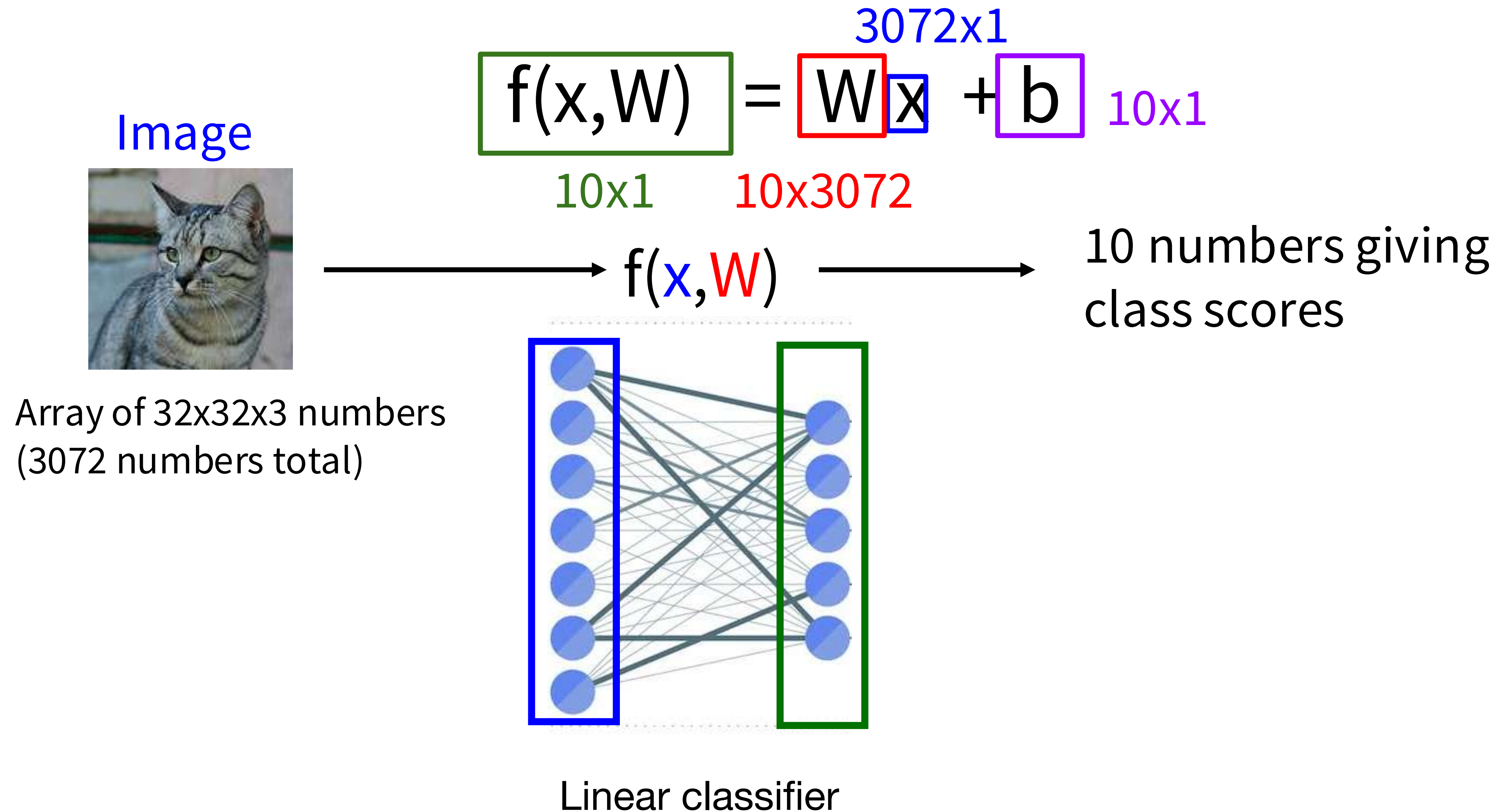
Parametric Approach: Linear Classifier



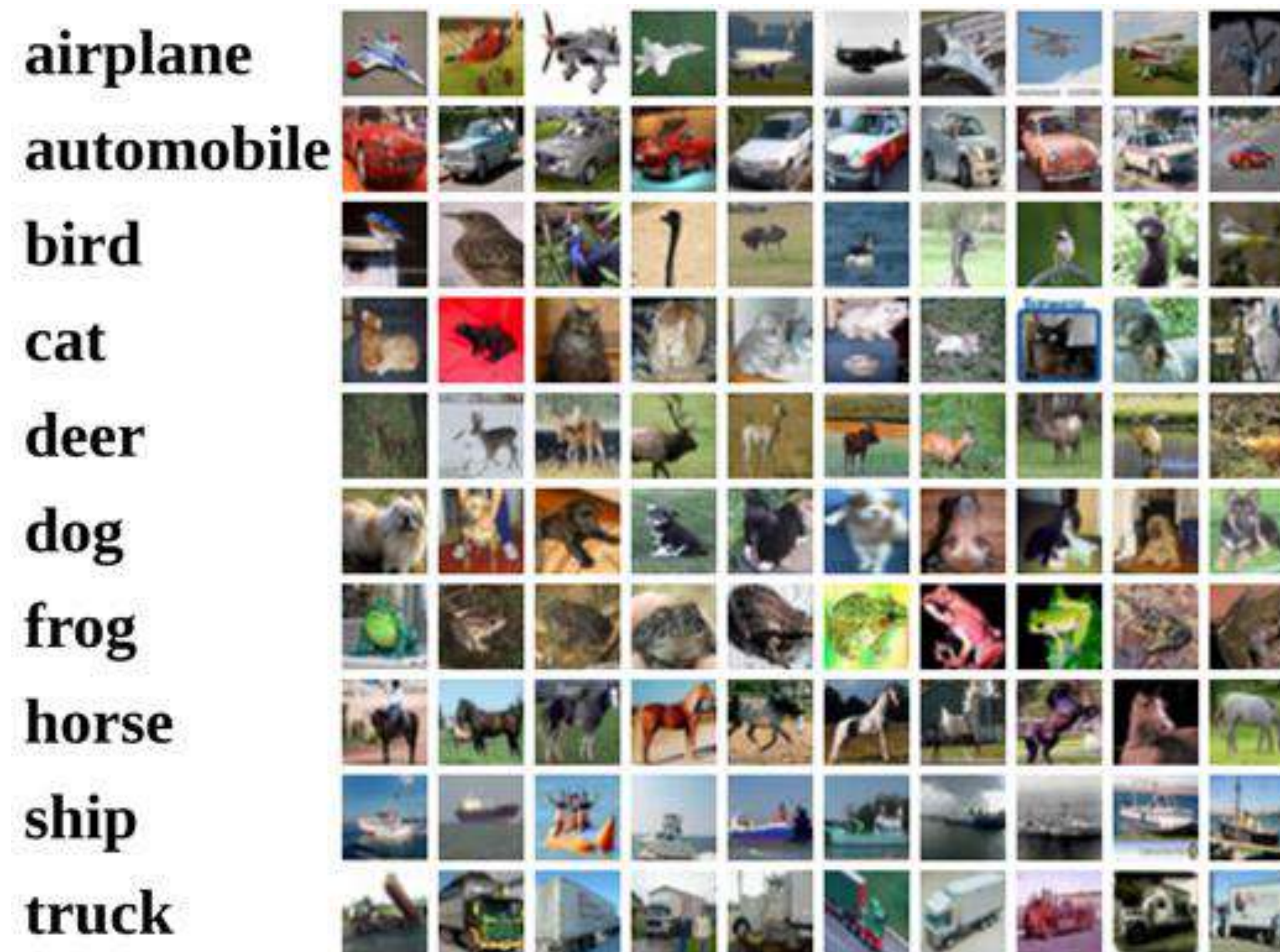
The weights control how the input affects each score.

The bias can move each class score up or down even when the input is zero

Parametric Approach: Linear Classifier



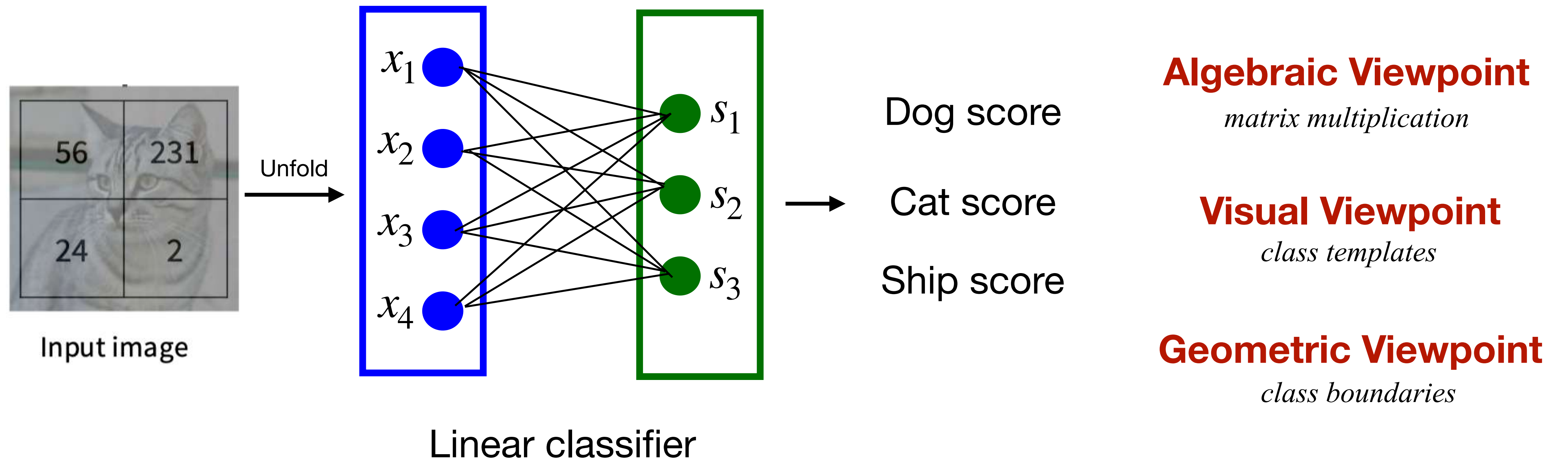
Recall CIFAR10



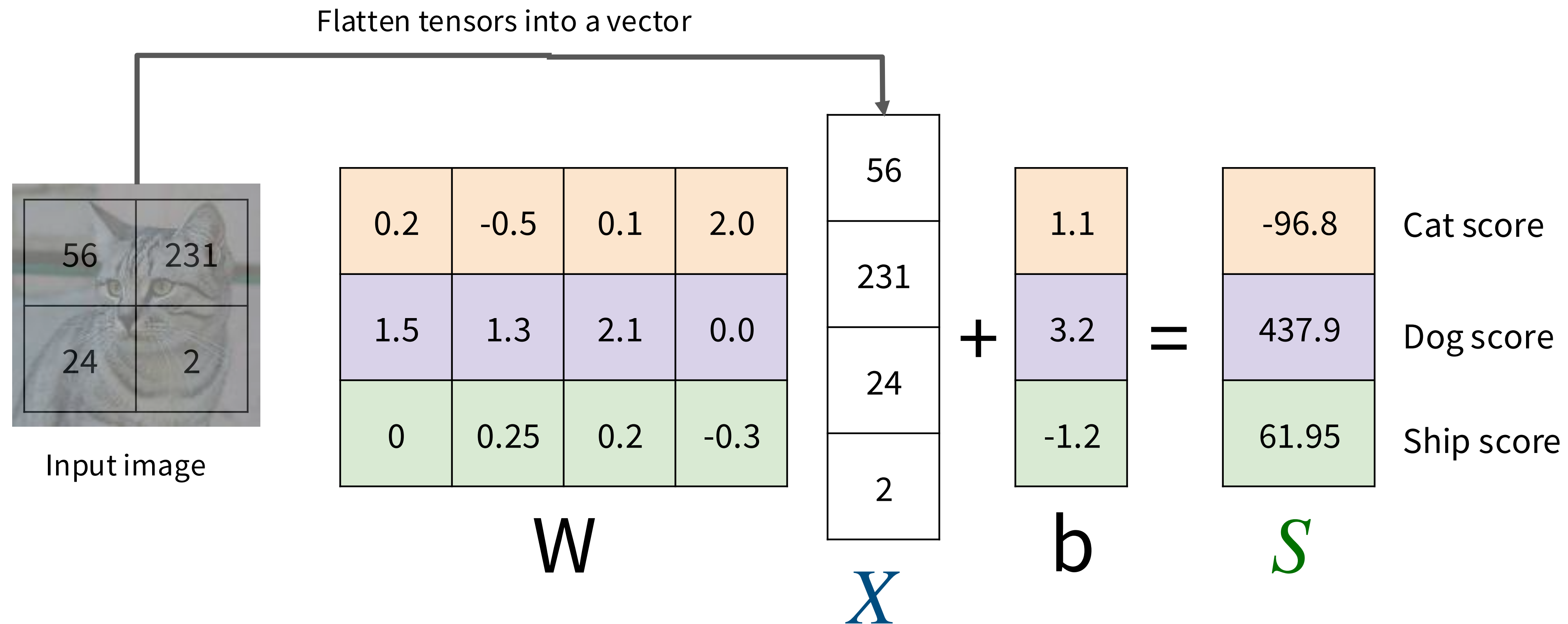
50,000 training images
each image is 32x32x3

10,000 test images.

Example with an image with 4 pixels and 3 classes



Interpreting a Linear Classifier: **Algebraic Viewpoint**

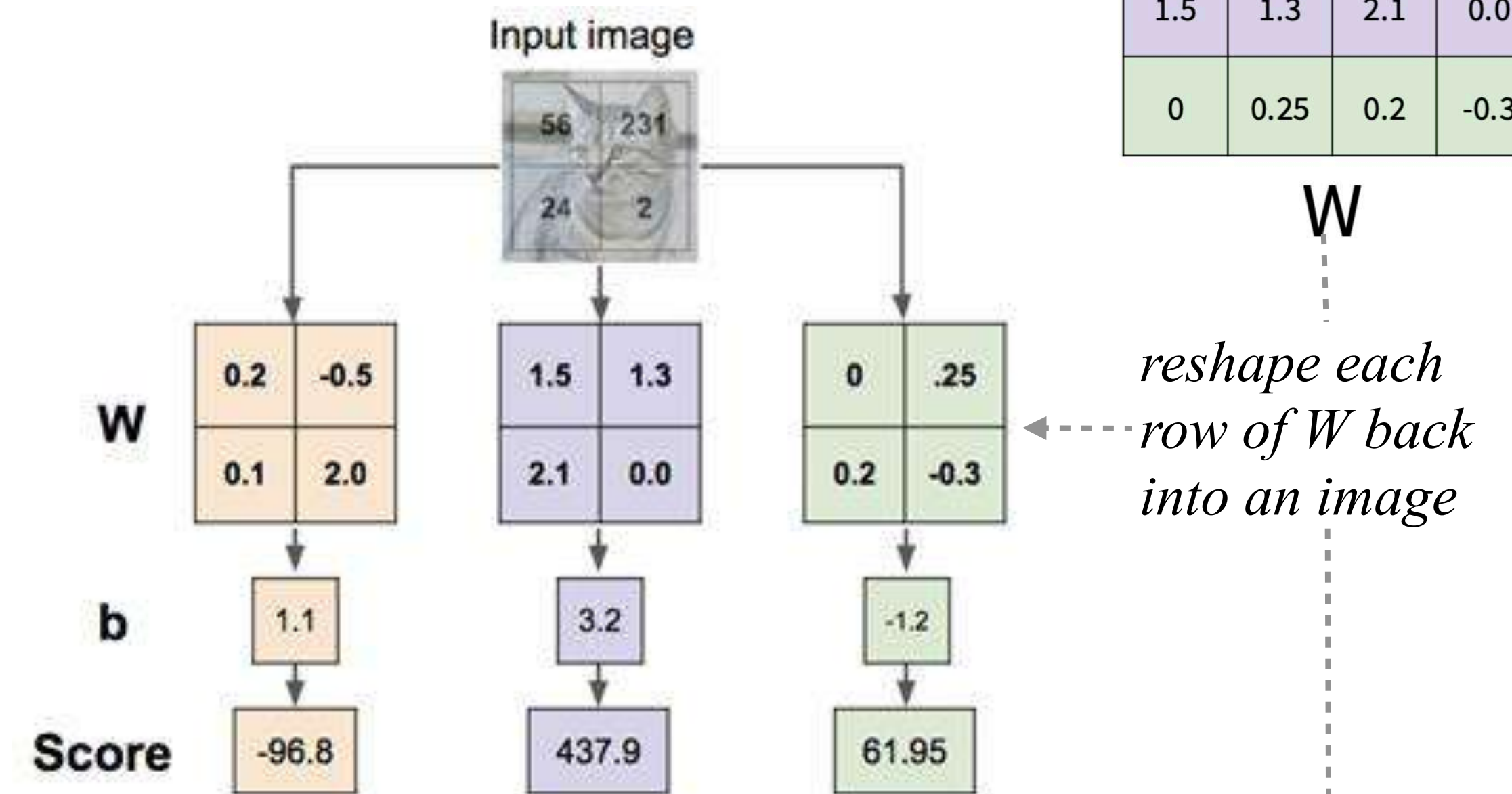
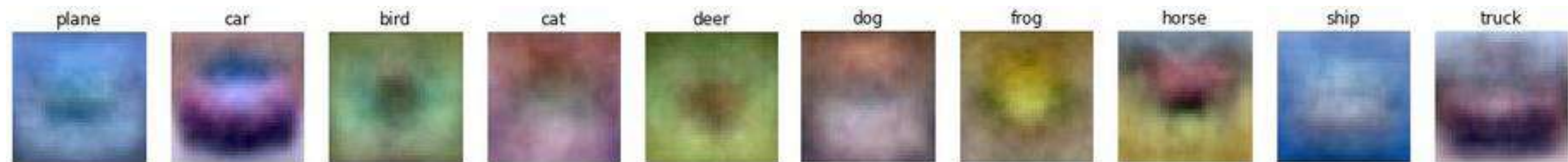
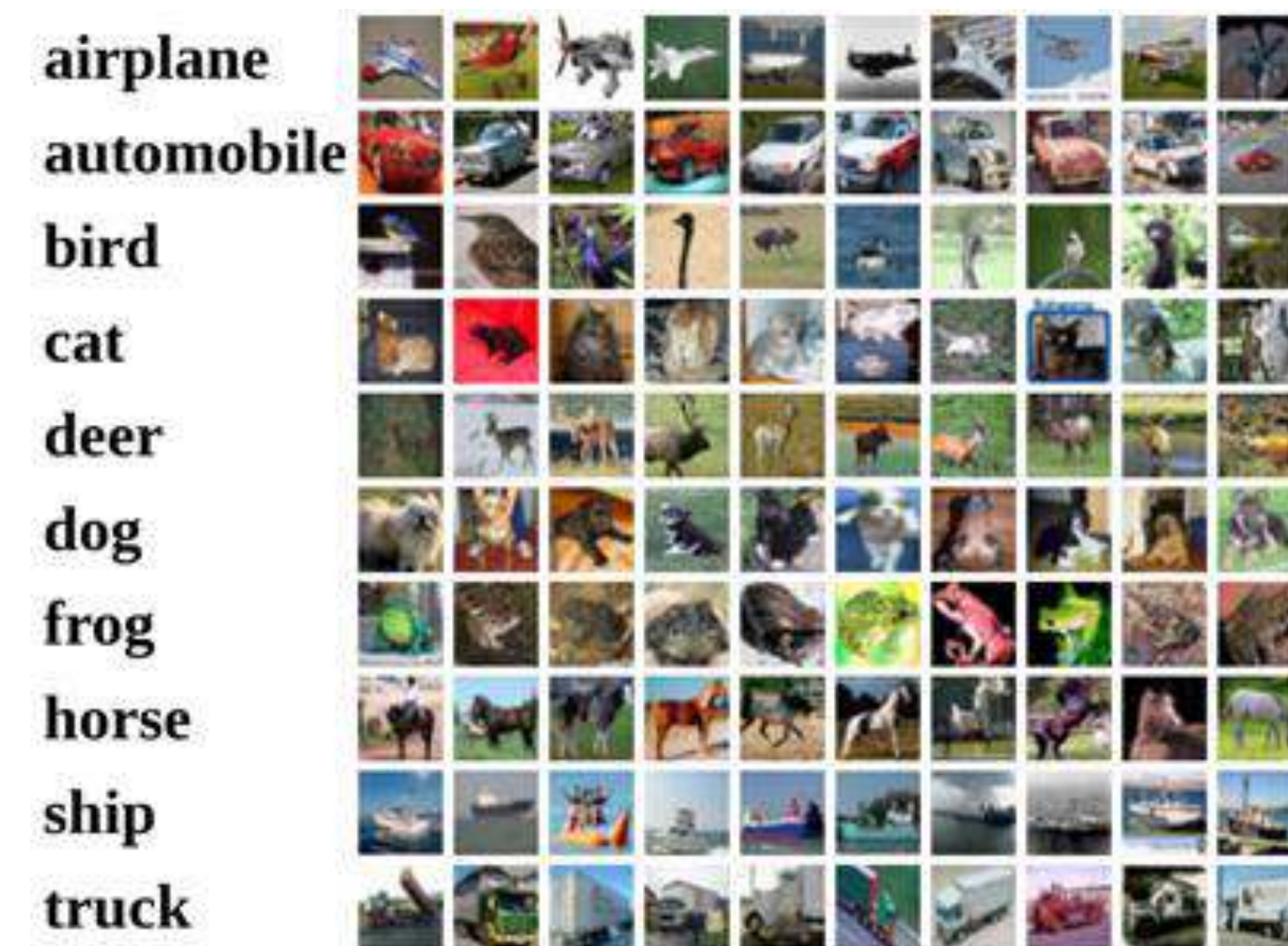


$$s_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1,$$

$$s_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_1,$$

$$s_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_1.$$

Interpreting a Linear Classifier: **Visual Viewpoint**

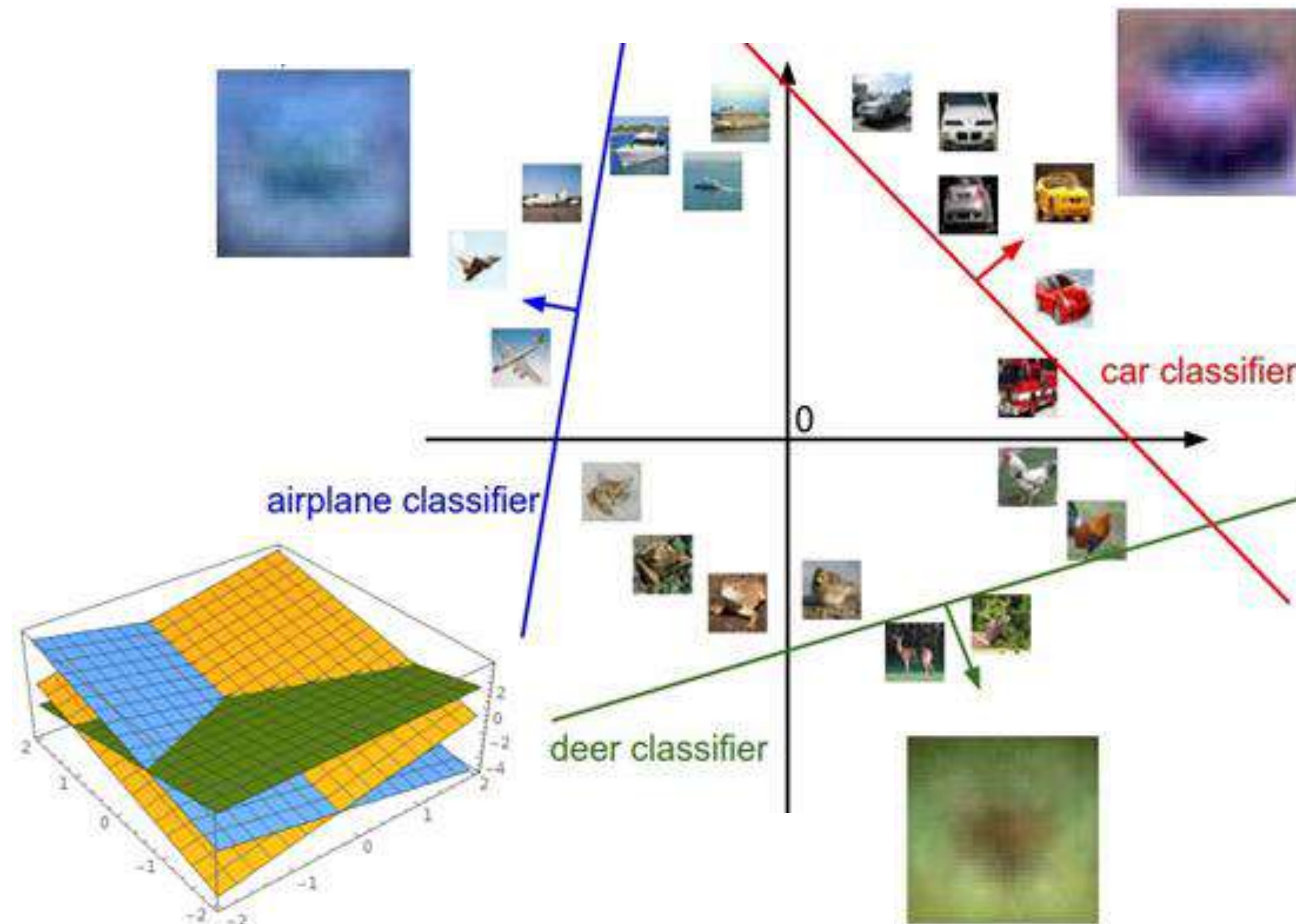


0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

Each row of W $\rightarrow$ class template

contain colors and patterns that often appear in this class images

Interpreting a Linear Classifier: **Geometric Viewpoint**



Plot created using [WolframCloud](#)

$$f(x, W) = Wx + b$$

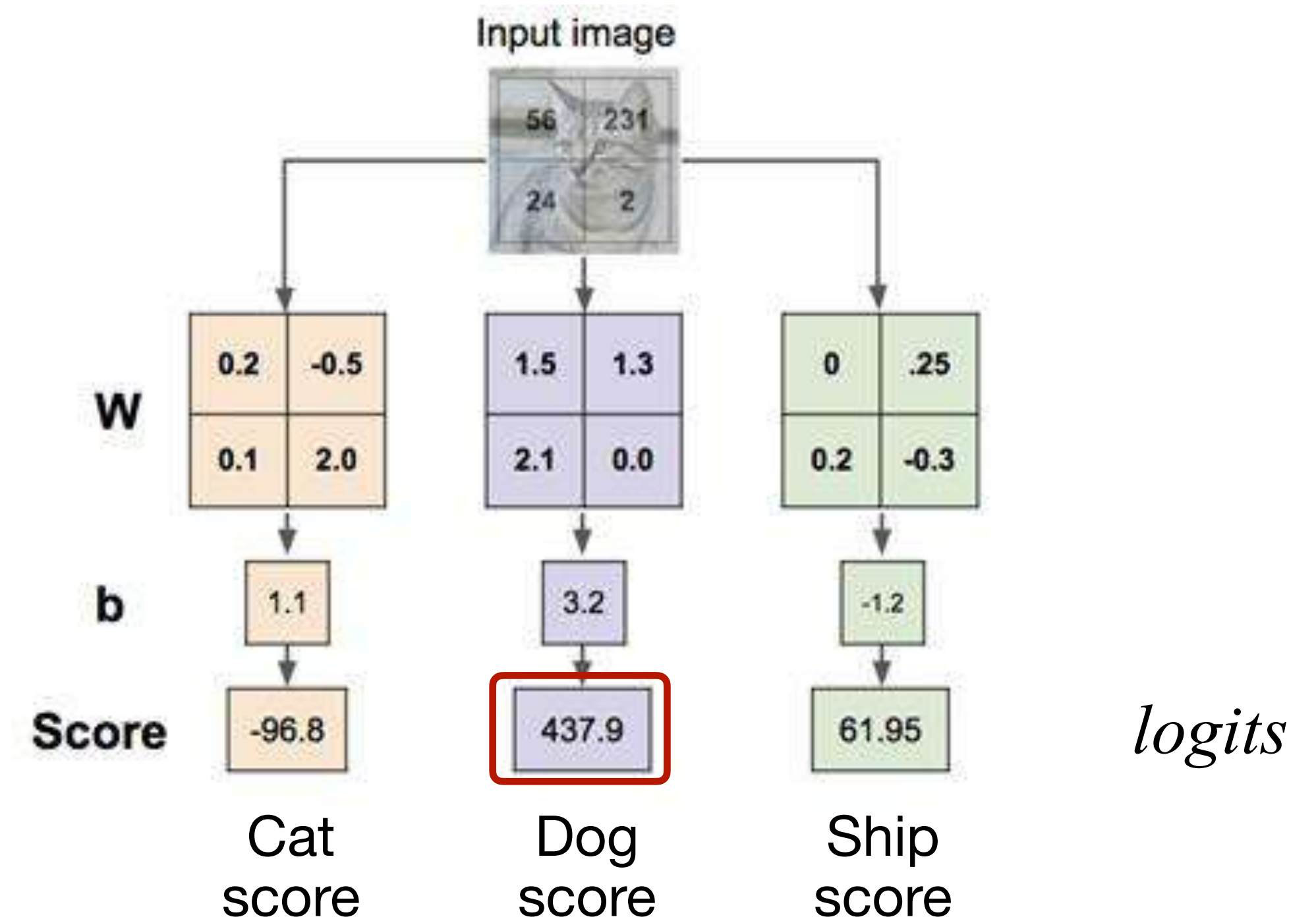
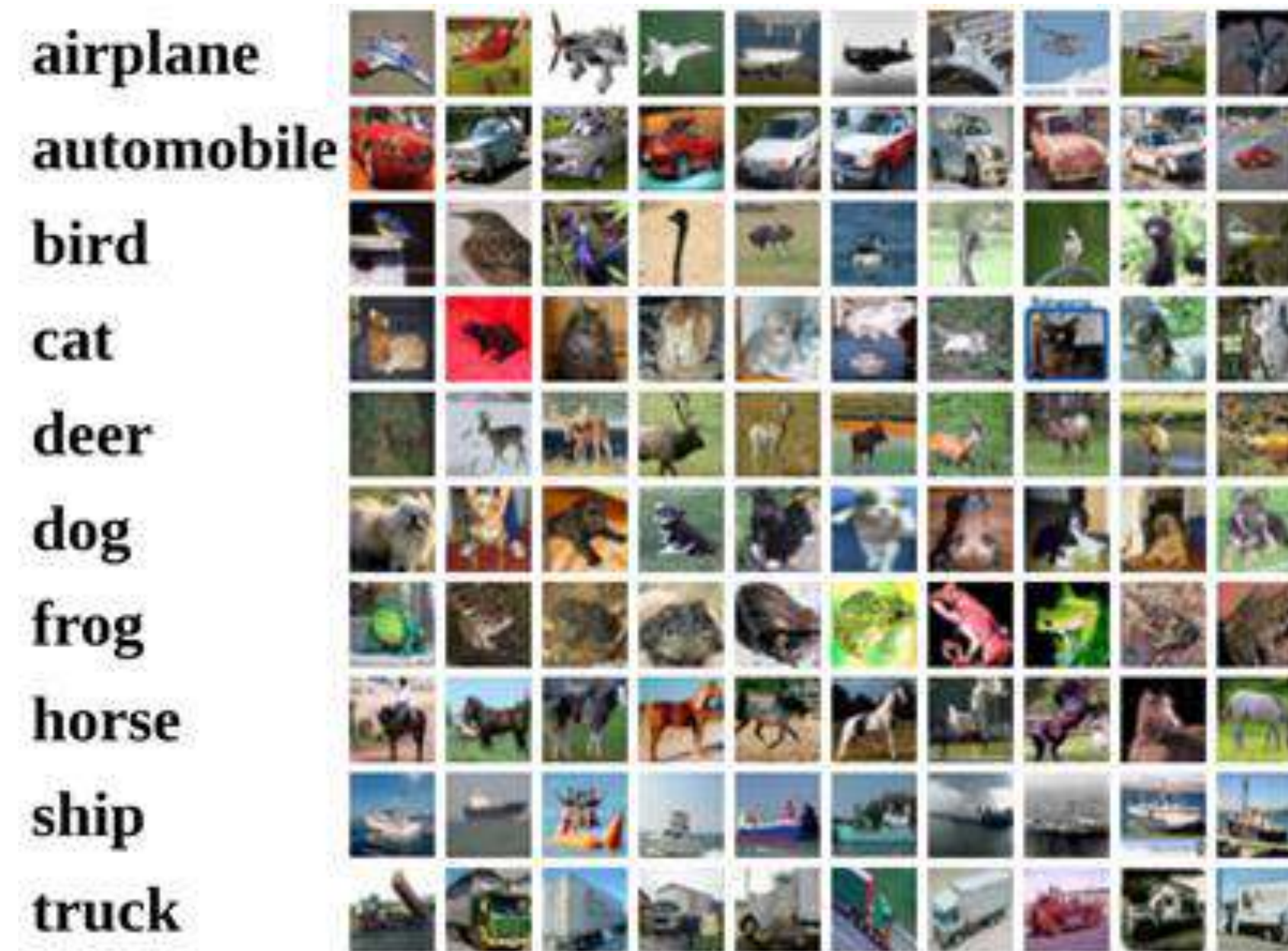


Array of 32x32x3 numbers
(3072 numbers total)

[Cat image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#)

*Every image is one point in a very high-dimensional space.
Each row of W defines a linear score surface in the high-dimensional space.*

Linear Classifier — Choose good weights W & b



Max score

X Wrong predicted label
Bad W & B

Linear Classifier — Choose good weights **W** & **b**



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

1. **Loss function:** *Define a loss function that quantifies our unhappiness with the predicted scores across the training data*
2. **Optimization:** *Come up with a way of efficiently finding the parameters that minimize the loss function*

[Cat image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#); [Car image](#) is [CC0 1.0](#) public domain; [Frog image](#) is in the public domain

Correct predictions

Linear Classifier — Choose good weights **W** & **b**



airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

[Cat image](#) by [Nik ita](#) is licensed under [CC-BY 2.0](#); [Car image](#) is [CC0 1.0](#) public domain; [Frog image](#) is in the public domain

Correct predictions

A loss function tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

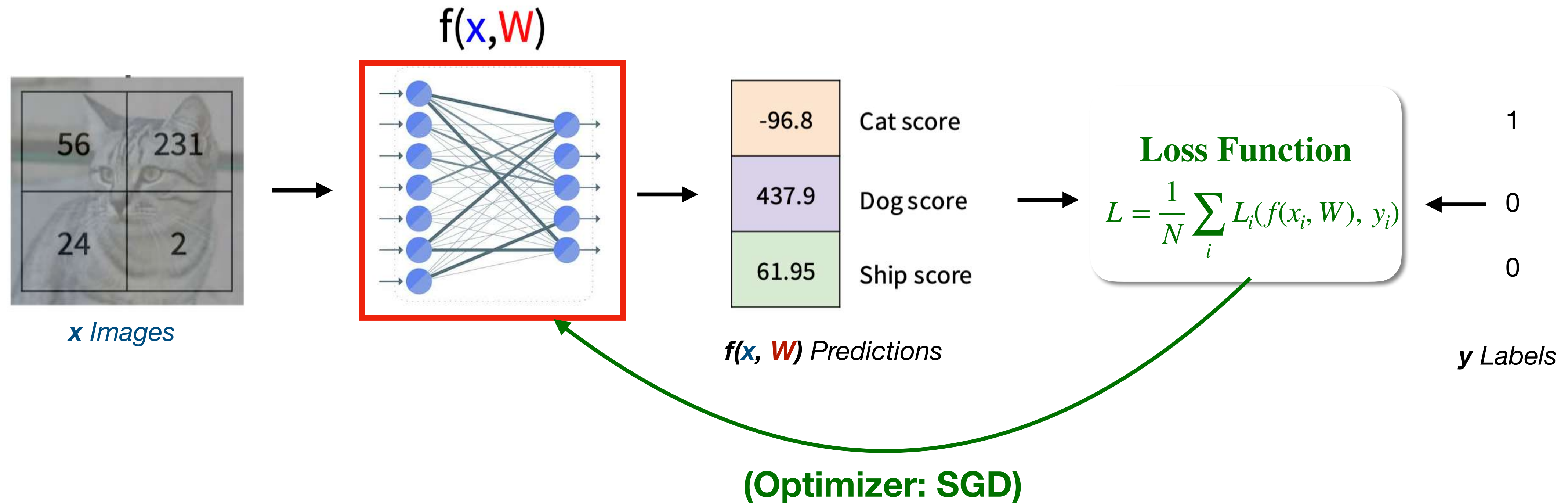
Where x_i is image and y_i is (integer) label

Loss over the dataset is an average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

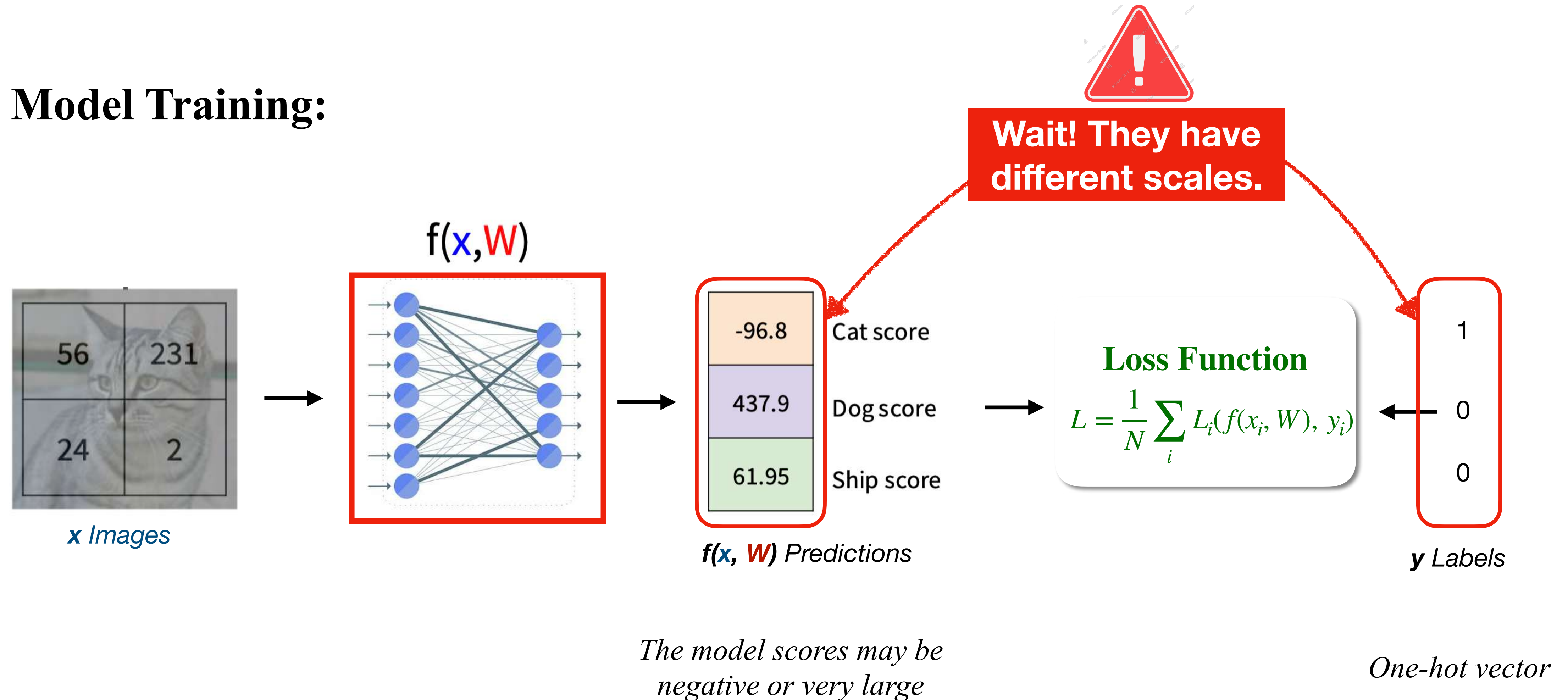
Linear Classifier — Choose good weights **W** & **b**

Model Training:



Linear Classifier — Choose good weights W & b

Model Training:



Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

*Probability measures how likely an event is to happen **on a scale from 0 (impossible) to 1 (certain)**.*

cat	3.2
car	5.1
frog	-1.7

$$p(cat|x) + p(car|x) + p(frog|x) = 1$$

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$s = [s_1, s_2, s_3]$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

cat

3.2

car

5.1

frog

-1.7

Unnormalized log-
Probabilities / logits

For class k , softmax function takes exponential of score s sub k and divides it by the sum of exp of every class score.

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

cat
car
frog

3.2

5.1

-1.7

exp

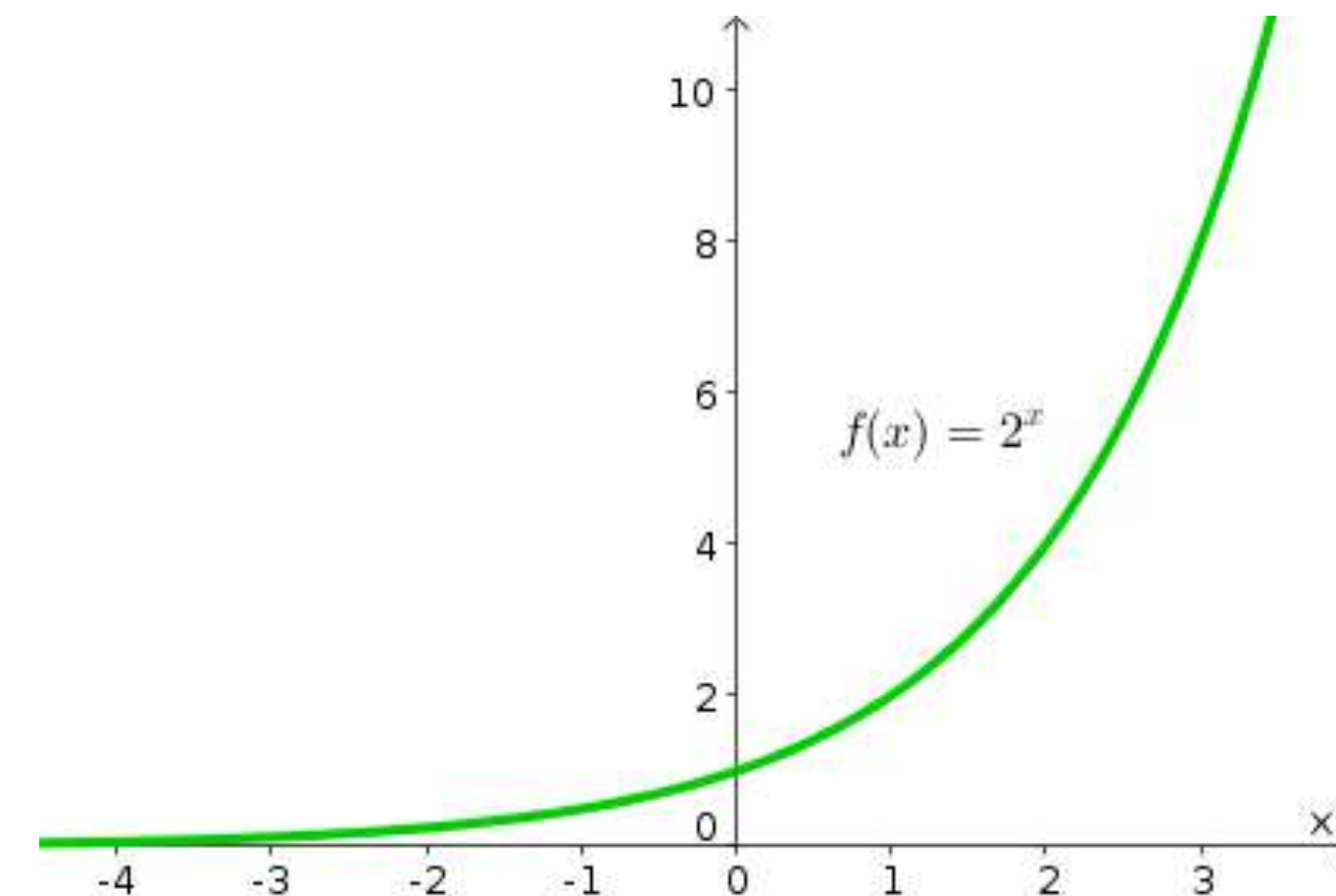
24.5

164.0

0.18

Unnormalized log-
Probabilities / logits

Unnormalized
Probabilities



Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

cat
car
frog

3.2
5.1
-1.7

exp

24.5
164.0
0.18

normalize

0.13
0.87
0.00

Unnormalized log-
Probabilities / logits

Unnormalized
Probabilities

probabilities

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

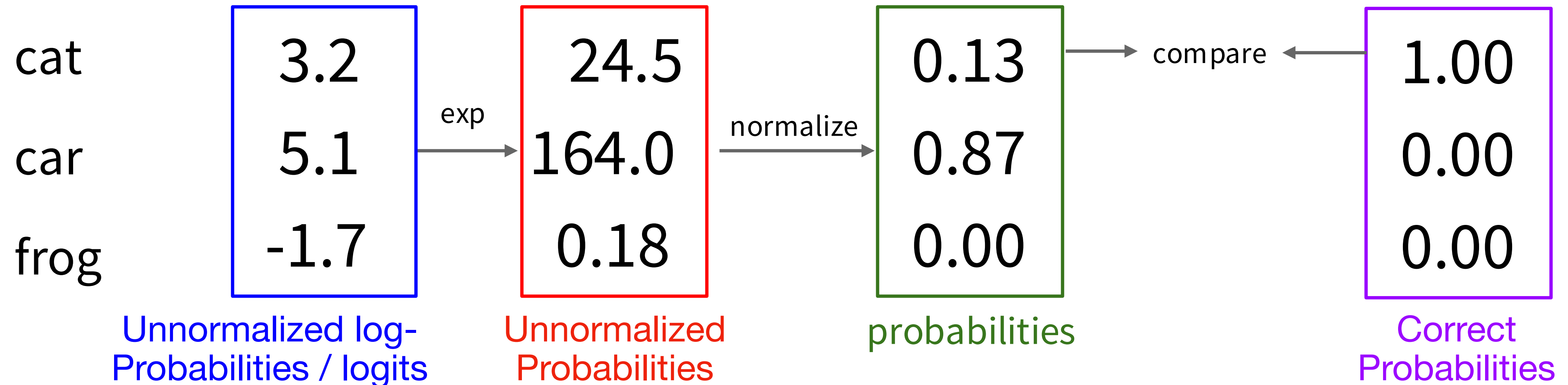
$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1



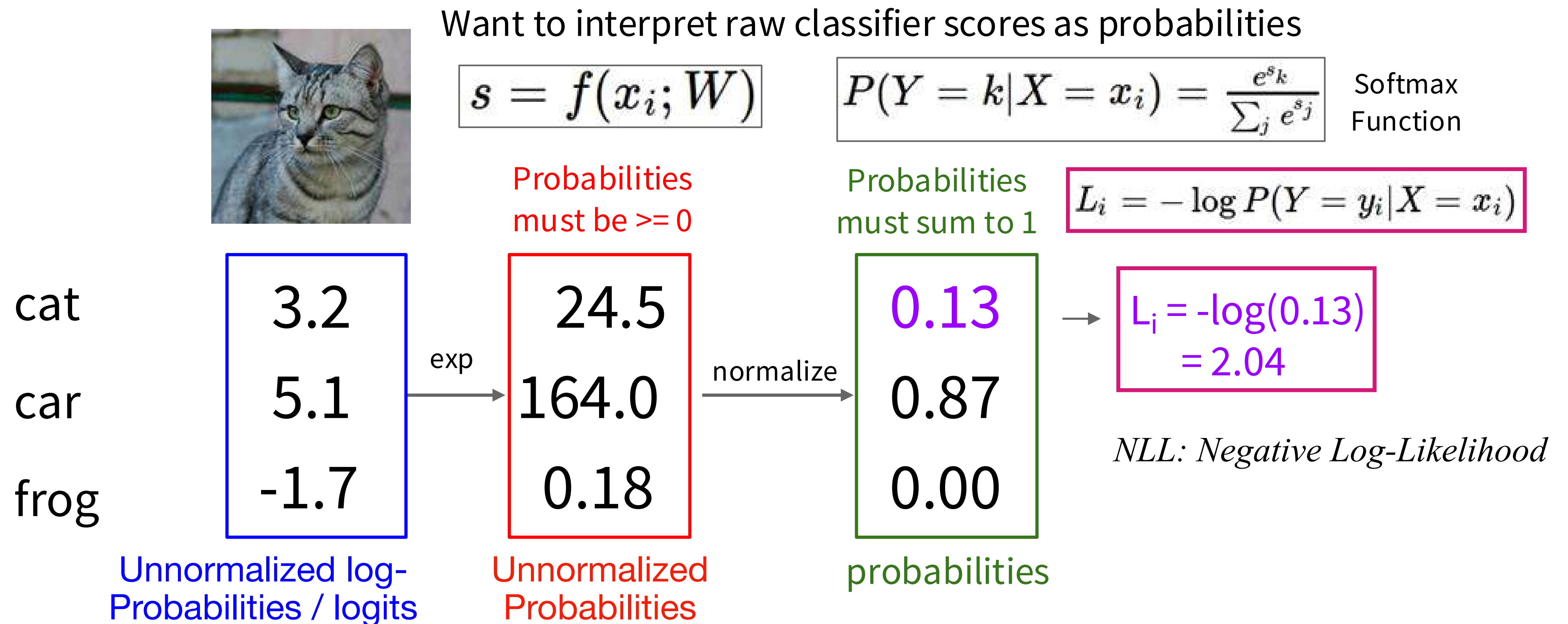
Straightforward idea:

(MLE) Maximum Likelihood Estimation to maximize the likelihood of the observed data

$$\max \sum_{k=1}^K P(Y = k | X = x_i) \cdot y_k$$

$$\max (0.13 \cdot 1 + 0.87 \cdot 0 + 0 \cdot 0) \in [0,1]$$

Softmax Classifier (Multinomial Logistic Regression)



Maximize MLE is equivalent to minimize NLL

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

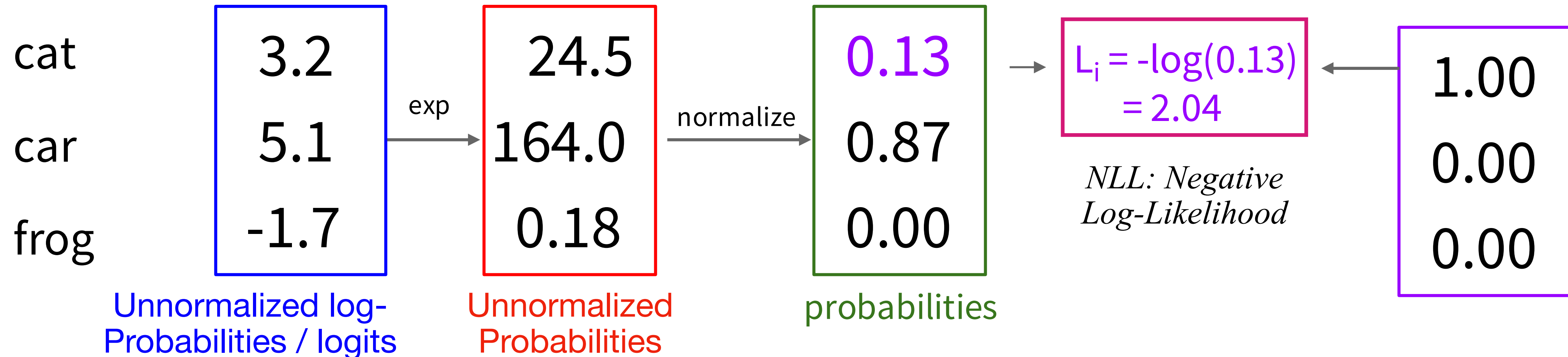
$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \quad \text{Softmax Function}$$

Probabilities
must be ≥ 0

Probabilities
must sum to 1

$$L_i = -\log P(Y = y_i|X = x_i)$$



Maximize MLE is equivalent to minimize NLL

Loss Function: Cross-Entropy Loss

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Q: How to define the loss function?}} = \frac{1}{N} \sum_{i=1}^N L_i(P(Y = y_i | X = x_i), y_i)$$

Q: How to define the loss function?

$$= -\frac{1}{N} \sum_{i=1}^N y_i \log P(Y = y_i | X = x_i) \quad \text{Negative Log-likelihood}$$

$$= -\sum_{i=1}^N y_i \log \frac{\exp(s_{y_i})}{\sum_j \exp(s_j)} \quad \text{Softmax}$$

Regularization

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}$$

Data loss: Model predictions should match training data

Learning noise or small details that do not repeat.

But we care about new test data.

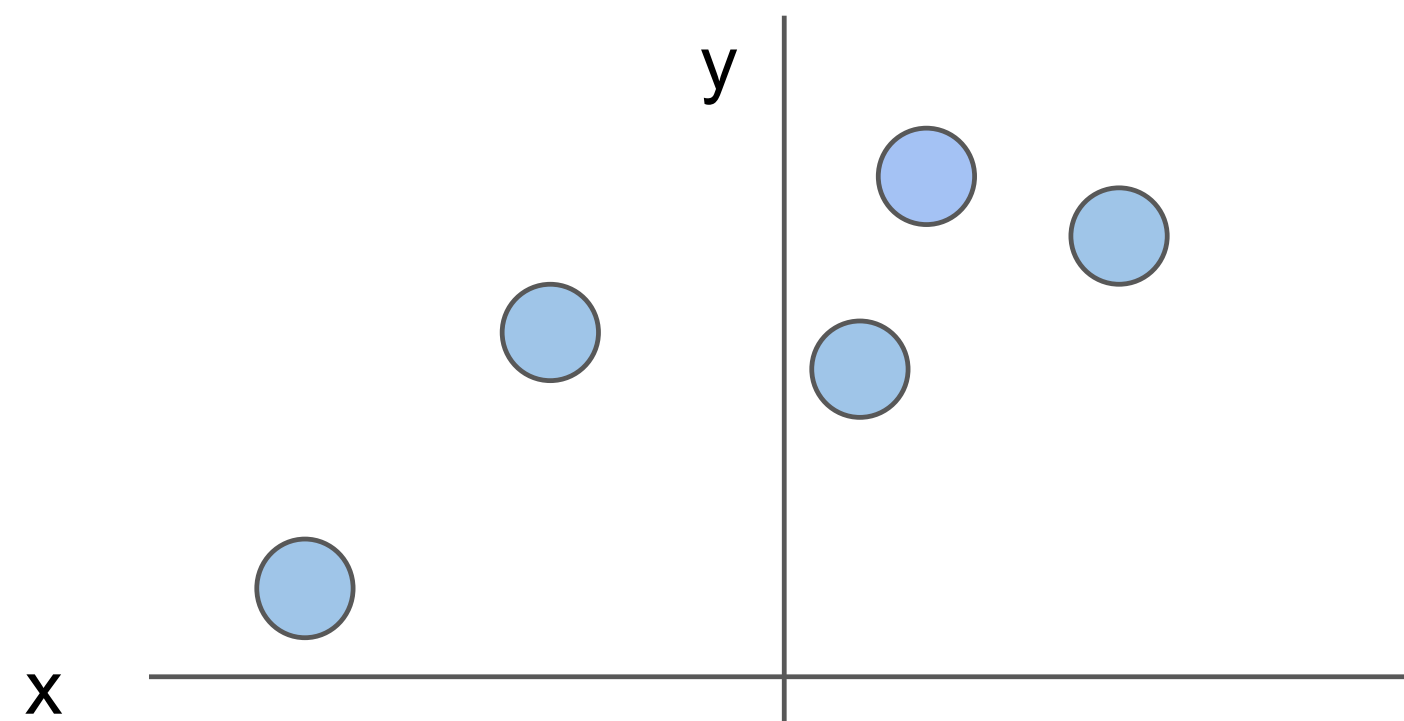


Q: Have we achieved good results so far?

Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data



Q: Have we achieved good results so far?

But we care about new test data.

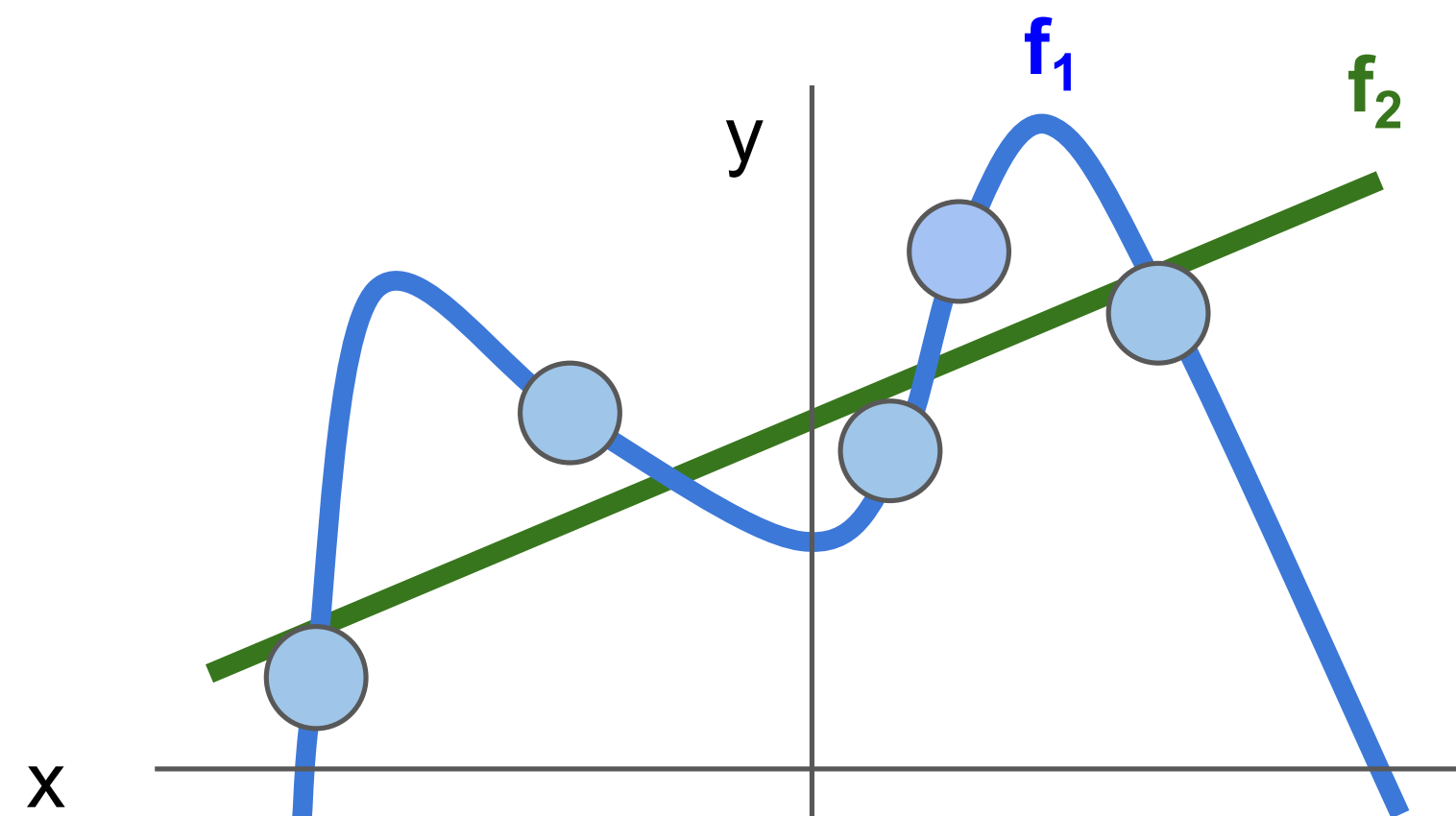
Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data



Q: Have we achieved good results so far?



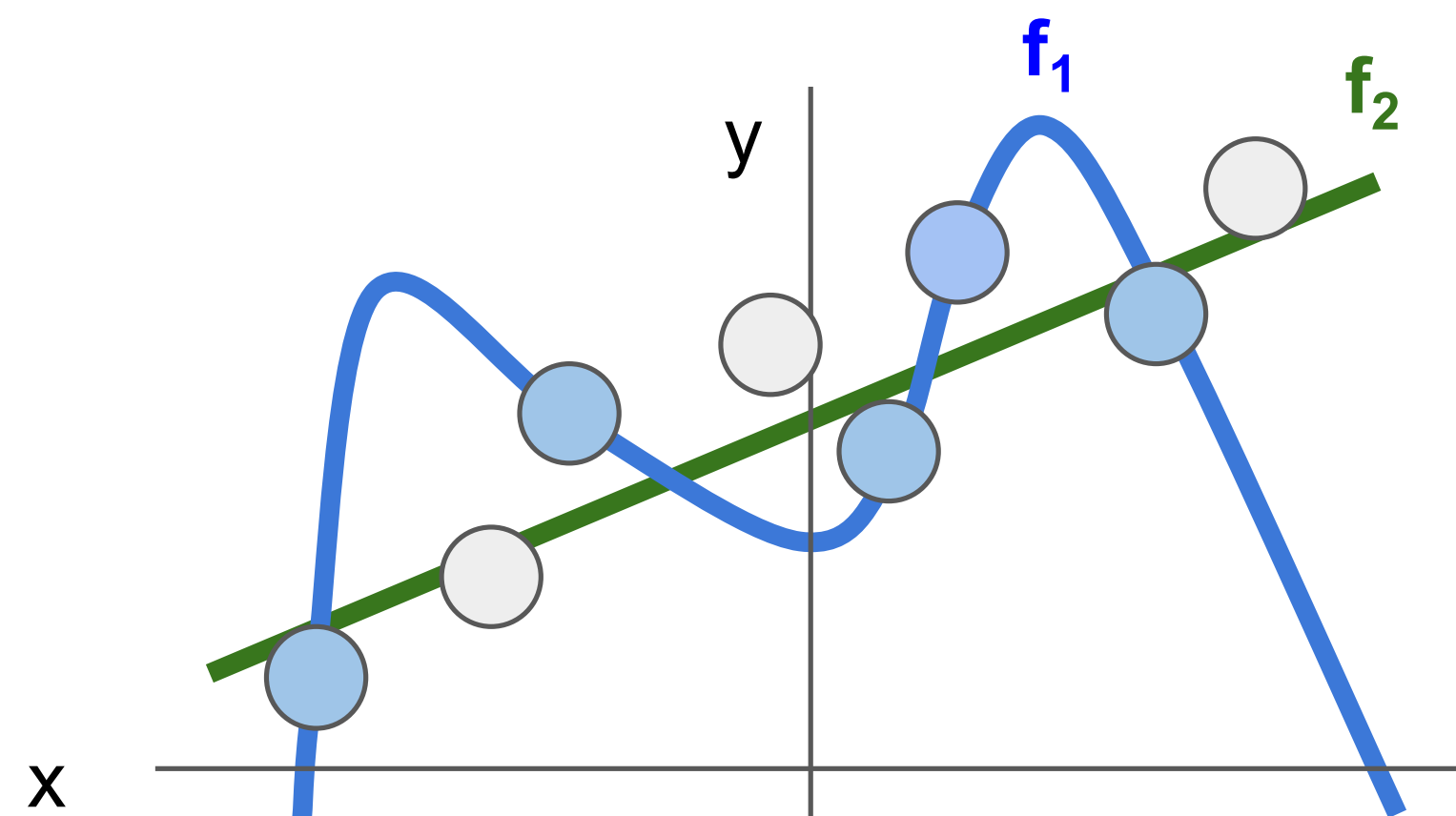
Training data $\rightarrow f_1$ or f_2

Training loss: $L(f_1) < L(f_2)$

Regularization

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}}$$

Data loss: Model predictions should match training data



Q: Have we achieved good results so far?

Testing data ○

Testing loss: $L(f_1) > L(f_2)$

$f_1 \rightarrow$ overfitting

The model performs very well on training data but poorly on new data.

Regularization

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Regularization

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Occam's Razor: Among multiple competing hypotheses, the simplest is the best, William of Ockham 1285-1347

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

the square of every weight

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

the absolute value of every weight

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

It encourages many weights to become 0.

The best choice depends on the model and the problem.

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

More complex:

Dropout

Batch normalization

Stochastic depth, fractional pooling, etc

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Why regularize?

- Express preferences over weights
- Make the model *simple* so it works on test data
- Improve optimization by adding curvature

Regularization: Expressing Preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will
the L2 regularizer prefer?

Regularization: Expressing Preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regularizer prefer?

L2 regularization likes to “spread out” the weights

Regularization: Expressing Preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

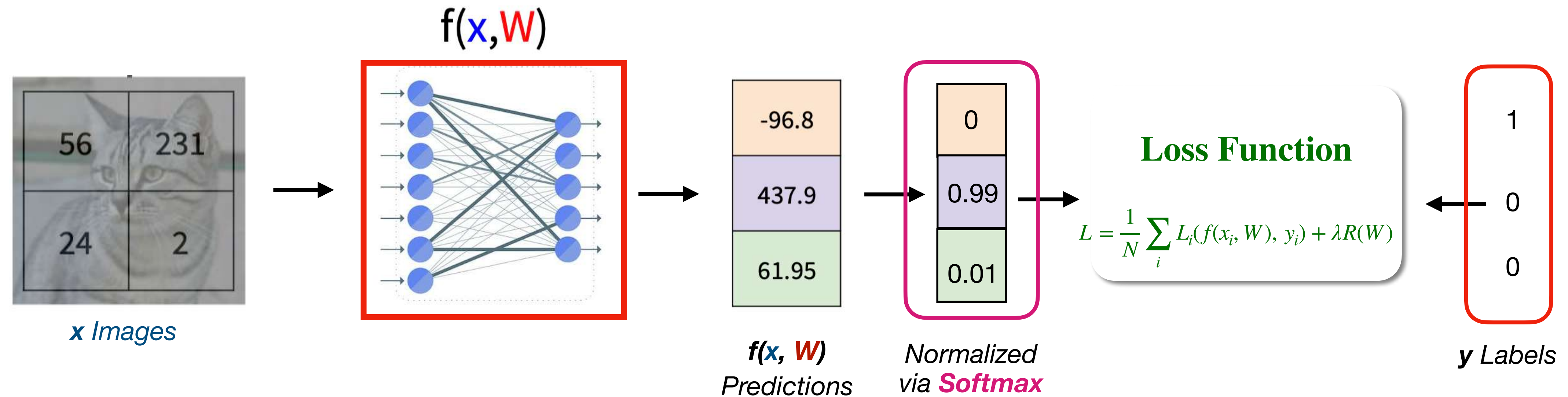
Which of w_1 or w_2 will the L2 regularizer prefer?

L2 regularization likes to “spread out” the weights

Which one would L1 regularization prefer?

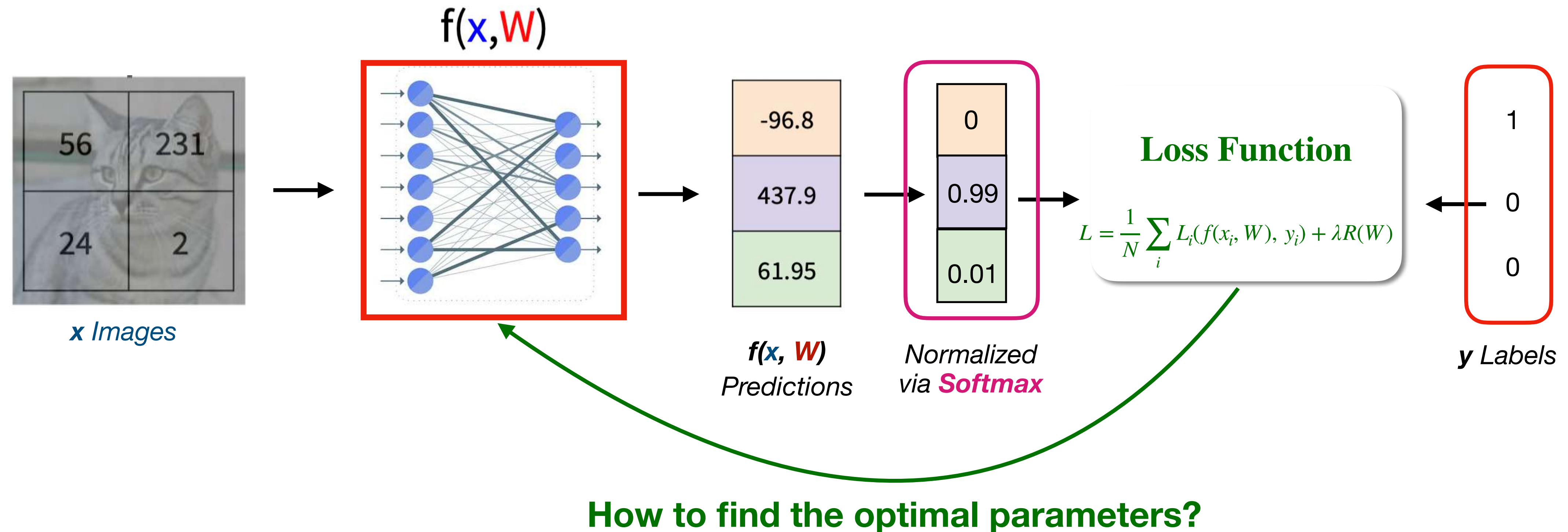
Linear Classifier

Model Training: Softmax Normalization + Cross-Entropy Loss + Regularization



Linear Classifier

Model Training: Softmax Normalization + Cross-Entropy Loss + Regularization



Next Lecture: Optimization

Q: How do we find the optimal path?

